

**APLICATIVO WEB DE GESTIÓN DE INVENTARIOS PARA LA TIENDA
MUNDOCOCO, TIENDA DE POSTRES, HELADOS Y BEBIDAS**

Cristian Camilo Díaz Ramírez

Carlos Daniel Camacho Valencia

Trabajo de grado para optar al título de Tecnólogo en Sistemas de Información

Director

Javier Cortés Carvajal



Institución Universitaria Antonio José Camacho

Facultad de Ingenierías

Programa Tecnología en Sistemas de Información

Santiago de Cali

2026

Tabla de contenido

	Pág.
Resumen.....	9
Abstract	11
Introducción	12
1. Planteamiento del Problema	13
1.1 Descripción del Problema	13
1.2 Formulación del Problema.....	14
1.3 Sistematización del Problema	14
1.4 Antecedentes	14
1.5 Justificación	15
2. Objetivos	17
2.1 Objetivo General.....	17
2.2 Objetivos Específicos.....	17
3. Marco Referencial.....	18
3.1 Marco Teórico.....	18
3.1.1 Gestión de Inventarios	18
3.1.2 Control Financiero y Cuadre de Caja.....	18
3.1.3 Sistemas de Información.....	19
3.1.4 Ingeniería de Requisitos.....	19
3.1.5 Metodologías Ágiles y Scrum.....	19
3.1.6 Arquitectura por Capas y Patrón MVC.....	20
3.1.7 PHP	20
3.1.8 Laravel	20
3.1.9 Filament, Livewire y Tailwind CSS	21
3.1.10 Sistema Gestor de Bases de Datos MySQL.....	21
3.1.11 Control de Acceso Basado en Roles	21
3.1.12 Pruebas de Software.....	21
3.1.13 Control de Versiones e Integración Continua	22
3.2 Marco Conceptual	22
3.3 Marco Contextual.....	23
3.4 Marco Legal	25
3.4.1 Ley 23 de 1982 – Derechos de Autor	25
3.4.2 Ley 603 de 2000 – Propiedad Intelectual en Software	25

3.4.3 Ley 1581 de 2012 y Decreto 1377 de 2013 – Protección de Datos Personales	25
3.4.4 Ley 1266 de 2008 – Habeas Data	26
3.4.5 Ley 527 de 1999 – Comercio Electrónico y Mensajes de Datos	26
3.4.6 Estándares de Ingeniería de Software	26
4. Método	27
4.1 Metodología de Desarrollo	27
4.1.1 Roles de Scrum	27
4.1.2 Artefactos y Eventos	28
4.1.3 Planificación de Sprints	28
4.2 Fase de Planificación	29
4.2.1 Levantamiento de Información	29
4.2.2 Requisitos Funcionales	30
4.2.3 Requisitos No Funcionales	31
4.2.4 Product Backlog.....	32
4.3 Fase de Diseño	33
4.3.1 Arquitectura del Sistema.....	33
4.3.2 Modelo Entidad-Relación	35
4.3.3 Diccionario de Datos.....	36
4.3.4 Diagrama de Clases.....	38
4.3.5 Diagramas de Casos de Uso.....	40
4.3.6 Casos de Uso Extendidos.....	43
4.3.7 Diagramas de Secuencia	50
4.3.8 Diagramas de Actividades	53
4.3.9 Diseño de Interfaces.....	57
4.4 Fase de Codificación.....	57
4.4.1 Entorno y Herramientas de Desarrollo	57
4.4.2 Organización del Código	58
4.4.3 Inicio de Sesión, Roles y Permisos	59
4.4.4 Productos y Categorías	61
4.4.5 Movimientos de Inventario y Trazabilidad.....	64
4.4.6 Ventas	66
4.4.7 Caja y Gastos	69
4.4.8 Escritorio, Alertas y Reportes	72
4.4.9 Seguridad, Auditoría y Respaldos.....	74

4.4.10 Protección de Datos Personales	76
4.4.11 Escalabilidad e Integración	77
4.5 Fase de Pruebas.....	77
4.5.1 Pruebas Unitarias y de Integración Automatizadas	78
4.5.2 Pruebas Funcionales.....	79
4.5.3 Pruebas de Rendimiento	83
4.5.4 Pruebas Heurísticas.....	84
4.5.5 Pruebas con el Usuario Final	85
4.6 Resultados Esperados.....	85
5. Resultados.....	87
6. Conclusiones y Observaciones	90
7. Recomendaciones	92
Referencias.....	93
Anexos	97

Lista de tablas

	Pág.
Tabla 1. <i>Líneas de producto de MundoCoco</i>	24
Tabla 2. <i>Actores de MundoCoco y su rol en el sistema</i>	24
Tabla 3. <i>Roles de Scrum en el proyecto</i>	27
Tabla 4. <i>Planificación de sprints e incrementos entregados</i>	28
Tabla 5. <i>Requisitos funcionales</i>	30
Tabla 6. <i>Requisitos no funcionales</i>	31
Tabla 7. <i>Product Backlog: historias de usuario</i>	32
Tabla 8. <i>Diccionario de datos, tabla productos</i>	36
Tabla 9. <i>Diccionario de datos, tabla ventas y venta_detalle</i>	37
Tabla 10. <i>Diccionario de datos, tabla caja</i>	37
Tabla 11. <i>Diccionario de datos, tabla movimientos_inventario</i>	37
Tabla 12. <i>Caso de uso extendido CU-01, Iniciar sesión</i>	43
Tabla 13. <i>Caso de uso extendido CU-02, Registrar usuario y asignar rol</i>	43
Tabla 14. <i>Caso de uso extendido CU-03, Registrar producto</i>	44
Tabla 15. <i>Caso de uso extendido CU-04, Registrar movimiento de inventario</i>	45
Tabla 16. <i>Caso de uso extendido CU-05, Registrar venta</i>	46
Tabla 17. <i>Caso de uso extendido CU-06, Anular venta</i>	46
Tabla 18. <i>Caso de uso extendido CU-07, Abrir caja</i>	47
Tabla 19. <i>Caso de uso extendido CU-08, Registrar gasto</i>	48
Tabla 20. <i>Caso de uso extendido CU-09, Cerrar caja</i>	48
Tabla 21. <i>Caso de uso extendido CU-10, Consultar y exportar reportes</i>	49
Tabla 22. <i>Tecnologías y herramientas utilizadas</i>	57
Tabla 23. <i>Pruebas unitarias automatizadas</i>	78
Tabla 24. <i>Pruebas de funcionalidad e integración automatizadas</i>	78
Tabla 25. <i>Casos de prueba funcionales</i>	80
Tabla 26. <i>Resultados de las pruebas de rendimiento</i>	84
Tabla 27. <i>Evaluación heurística de usabilidad</i>	84
Tabla 28. <i>Cumplimiento de los requisitos</i>	88
Tabla 29. <i>Recorrido diario recomendado del manual de usuario</i>	102
Tabla 30. <i>Trazabilidad de requisitos, casos funcionales y pruebas automatizadas</i>	103

Lista de figuras

	Pág.
Figura 1. <i>Arquitectura por capas del sistema</i>	34
Figura 2. <i>Diagrama entidad-relación</i>	35
Figura 3. <i>Diagrama de clases del dominio</i>	39
Figura 4. <i>Diagrama de clases de la capa de servicios</i>	39
Figura 5. <i>Diagrama de casos de uso, gestión de usuarios y seguridad</i>	40
Figura 6. <i>Diagrama de casos de uso, gestión de inventario</i>	41
Figura 7. <i>Diagrama de casos de uso, ventas</i>	42
Figura 8. <i>Diagrama de casos de uso, caja, gastos y reportes</i>	42
Figura 9. <i>Diagrama de secuencia, iniciar sesión</i>	50
Figura 10. <i>Diagrama de secuencia, registrar venta</i>	51
Figura 11. <i>Diagrama de secuencia, cerrar caja</i>	52
Figura 12. <i>Diagrama de secuencia, registrar movimiento de inventario</i>	52
Figura 13. <i>Diagrama de secuencia, consultar y exportar reportes</i>	53
Figura 14. <i>Diagrama de actividades, registrar venta</i>	54
Figura 15. <i>Diagrama de actividades, apertura y cuadre de caja</i>	55
Figura 16. <i>Diagrama de actividades, registrar movimiento de inventario</i>	56
Figura 17. <i>Conversión exacta de valores monetarios a centavos</i>	59
Figura 18. <i>Interfaz de inicio de sesión</i>	60
Figura 19. <i>Asignación de permisos a los roles Administrador, Operario y Consultor</i>	61
Figura 20. <i>Interfaz de gestión de usuarios</i>	61
Figura 21. <i>Interfaz de listado de productos</i>	62
Figura 22. <i>Validación del precio de venta en el formulario de producto</i>	62
Figura 23. <i>Interfaz de registro de producto</i>	63
Figura 24. <i>Interfaz de categorías</i>	63
Figura 25. <i>Cálculo del stock total a partir del historial de movimientos (RF05)</i>	64
Figura 26. <i>Interfaz del historial de movimientos de inventario</i>	65
Figura 27. <i>Interfaz de registro de movimiento de inventario por lote</i>	65
Figura 28. <i>Registro transaccional de una venta</i>	66
Figura 29. <i>Bloqueo de productos y validación de stock agregado</i>	67
Figura 30. <i>Interfaz de listado de ventas con estado y método de pago</i>	67
Figura 31. <i>Interfaz de registro de venta</i>	68
Figura 32. <i>Diálogo de anulación de venta</i>	69
Figura 33. <i>Cierre de caja con cálculo del saldo teórico y la diferencia (RF11)</i>	70
Figura 34. <i>Interfaz de cajas con totales por método de pago</i>	71
Figura 35. <i>Diálogo de cierre de caja</i>	71
Figura 36. <i>Interfaz de gastos</i>	72
Figura 37. <i>Escritorio con indicadores y productos con stock crítico</i>	73
Figura 38. <i>Interfaz de reportes</i>	74
Figura 39. <i>Reporte de valorización de inventario exportado a PDF</i>	74
Figura 40. <i>Registro de una operación en la auditoría</i>	75
Figura 41. <i>Tareas programadas de respaldo diario y limpieza de auditoría</i>	75
Figura 42. <i>Interfaz de auditoría</i>	76
Figura 43. <i>Middleware que exige el consentimiento para el tratamiento de datos</i>	77

Figura 44. <i>Validación de credenciales inválidas en el inicio de sesión</i>	81
Figura 45. <i>Validación de campos obligatorios en el registro de venta</i>	82
Figura 46. <i>Escritorio del rol Operario: solo ventas y consulta de inventario</i>	82
Figura 47. <i>Escritorio del rol Consultor: solo indicadores y reportes</i>	83
Figura 48. <i>Acceso denegado al Operario en el módulo de usuarios</i>	83

Lista de anexos

Anexo 1. Guía de preguntas para el levantamiento de información

Anexo 2. Encuesta de satisfacción del usuario final

Anexo 3. Carta de autorización de la empresa MundoCoco

Anexo 4. Resumen del manual de usuario

Anexo 5. Matriz de trazabilidad entre requisitos y pruebas

Resumen

El presente trabajo de grado describe el análisis, diseño, desarrollo y validación de un aplicativo web de gestión de inventarios para MundoCoco, tienda de postres, helados, bebidas y productos derivados del coco ubicada en Santiago de Cali. Antes del proyecto, la tienda registraba su inventario, sus ventas y el cuadre de caja en hojas de papel y archivos básicos de Excel diligenciados por la persona de turno, lo que producía errores de registro, diferencias en el cierre de caja, ausencia de información en tiempo real y pérdidas no detectadas.

La solución se construyó con la metodología ágil Scrum, organizada en sprints con entregas incrementales validadas por la administración de la tienda. El sistema se desarrolló en PHP 8.3 con el framework Laravel 13 y el panel administrativo Filament 5, sobre una base de datos MySQL, siguiendo una arquitectura por capas en la que toda la lógica de negocio reside en servicios transaccionales. El aplicativo integra los módulos de productos y categorías, movimientos de inventario con trazabilidad completa, ventas con descuento automático de stock, control de caja diaria con categorización de métodos de pago (efectivo, Nequi, transferencia y tarjeta), registro de gastos, alertas de stock mínimo con sugerencia de reorden, reportes exportables a PDF, Excel y CSV, control de usuarios por roles, auditoría de operaciones críticas, respaldo diario de la base de datos y consentimiento para el tratamiento de datos personales conforme a la Ley 1581 de 2012.

La calidad del software se verificó con una suite de 160 pruebas automatizadas (566 aserciones) escritas con Pest, ejecutadas en integración continua, además de pruebas funcionales, de rendimiento y heurísticas de usabilidad. Los resultados evidencian el cumplimiento de los doce requisitos funcionales y los diez requisitos no funcionales definidos con el cliente.

Palabras clave: gestión de inventarios, cuadro de caja, aplicativo web, Laravel, Scrum, trazabilidad, pequeña empresa.

Abstract

This undergraduate project describes the analysis, design, development and validation of a web-based inventory management application for MundoCoco, a store in Santiago de Cali that sells desserts, ice cream, beverages and coconut products. Before the project, the store recorded its inventory, sales and daily cash reconciliation on paper sheets and basic Excel files filled in by the employee on shift, which caused recording errors, cash discrepancies, lack of real-time information and undetected losses.

The solution was built using the Scrum agile methodology, organized in sprints with incremental deliveries validated by the store management. The system was developed in PHP 8.3 with the Laravel 13 framework and the Filament 5 admin panel on a MySQL database, following a layered architecture in which all business logic lives in transactional services. The application integrates product and category management, fully traceable inventory movements, sales with automatic stock deduction, daily cash control with payment method categorization (cash, Nequi, bank transfer and card), expense tracking, minimum-stock alerts with reorder suggestions, reports exportable to PDF, Excel and CSV, role-based user control, auditing of critical operations, daily database backups and personal data processing consent in accordance with Colombian Law 1581 of 2012.

Software quality was verified with a suite of 160 automated tests (566 assertions) written with Pest and executed in continuous integration, together with functional, performance and heuristic usability tests. The results show compliance with the twelve functional and ten non-functional requirements defined with the client.

Keywords: inventory management, cash reconciliation, web application, Laravel, Scrum, traceability, small business.

Introducción

MundoCoco es una tienda de la ciudad de Santiago de Cali especializada en la venta de helados artesanales, bebidas, aceites de coco y productos derivados del coco. Como ocurre en muchos pequeños negocios locales, su operación diaria dependía de registros manuales: el inventario se anotaba en hojas de papel con tablas en blanco, las ventas se sumaban a mano y el cuadre de caja se realizaba en el reverso de la misma hoja, sin validación alguna. Estos registros eran entregados posteriormente al propietario para su procesamiento, lo que retrasaba la información y multiplicaba las oportunidades de error.

Esta forma de trabajo generaba diferencias recurrentes entre el dinero esperado y el dinero contado al cierre de la jornada, desconocimiento del stock real de cada producto, ventas perdidas por agotamiento no detectado y una carga adicional para el personal de turno, que en ocasiones debía extender su jornada para completar el inventario. Frente a este panorama, el proyecto propuso y desarrolló un aplicativo web que centraliza el control de stock, el registro de ventas y el cuadre de caja, incorporando la categorización de los métodos de pago utilizados por la tienda.

El documento se organiza de la siguiente manera: el capítulo 1 presenta el planteamiento del problema, los antecedentes y la justificación; el capítulo 2, los objetivos; el capítulo 3, el marco referencial (teórico, conceptual, contextual y legal); el capítulo 4 describe el proceso de desarrollo bajo la metodología Scrum, desde la planificación y el levantamiento de requisitos hasta el diseño, la codificación y las pruebas; los capítulos 5, 6 y 7 exponen los resultados, las conclusiones y las recomendaciones. Finalmente se incluyen las referencias y los anexos.

1. Planteamiento del Problema

1.1 Descripción del Problema

MundoCoco opera con un sistema de gestión de inventarios completamente manual, basado en cuadernos, hojas de papel y archivos básicos de Excel. El registro de los productos se realiza de forma física, en hojas con tablas en blanco que se diligencian a mano por la persona que se encuentra de turno. Este procedimiento, además de ser demorado, genera errores frecuentes, duplicidad de información y pérdida de datos, lo que impide mantener un inventario claro, actualizado y confiable.

El proceso de cuadre de caja se lleva a cabo en la misma hoja de registro, generalmente en el reverso del documento, sin ningún tipo de control automatizado. Al finalizar la jornada no se cuenta con información precisa sobre los movimientos financieros realizados: los pagos recibidos por transferencia bancaria o Nequi, los pagos en efectivo, los descuentos aplicados o las devoluciones. Tampoco existe un registro formal de los gastos operativos, en especial de los relacionados con la compra de materia prima. Como consecuencia, en el cuadre diario suelen presentarse diferencias entre el dinero que debería haber y el que realmente hay; aunque a veces son pequeñas, su acumulación afecta el control financiero y genera incertidumbre sobre el manejo de los recursos.

De acuerdo con el análisis realizado en el anteproyecto, las consecuencias de esta situación incluyen pérdidas económicas estimadas entre el 15 % y el 30 % de la rentabilidad potencial, un incremento de los tiempos operativos entre el 30 % y el 40 %, ventas perdidas por desconocimiento del stock real, acumulación de productos de baja rotación y la imposibilidad de escalar el negocio. La problemática evidencia deficiencias en la captura, el procesamiento, el almacenamiento, la calidad y la distribución de la información crítica para la operación.

1.2 Formulación del Problema

¿Cómo afecta la ausencia de un sistema automatizado de gestión de inventarios, que integre control de stock, registro de ventas, cuadre de caja y categorización de métodos de pago, en la rentabilidad y la eficiencia operativa de la tienda MundoCoco ubicada en Cali, Colombia?

1.3 Sistematización del Problema

- ¿En qué medida la gestión manual del inventario inicial y final impacta la precisión del control de stock en MundoCoco?
- ¿Cómo afecta la falta de información en tiempo real el número de ventas perdidas por agotamiento de productos?
- ¿Qué efecto tienen los procesos manuales de cuadre de caja y registro de ventas en el tiempo promedio de cierre de jornada y en la exactitud del reporte financiero?
- ¿Cómo influye la ausencia de categorización de métodos de pago en la conciliación financiera y en el análisis de ventas por canal (efectivo, transferencias bancarias, tarjetas)?

1.4 Antecedentes

La adopción de sistemas de información en pequeñas empresas ha sido objeto de varios trabajos de grado que sirven como referencia para este proyecto. Amaya Beltrán y Preciado Guerrero (2013), en la Universidad EAN, desarrollaron una solución web para optimizar los procesos de inventario y cotización de la perfumería El Mundo de Tu Belleza, un negocio de comercio minorista con necesidades similares a las de MundoCoco: control de existencias de productos de alta rotación y agilidad en la atención al cliente.

Hernández, Rodríguez y Gómez (2021) presentaron el diseño e implementación de un sistema de gestión de inventarios para la empresa Técnitaller S.A.S., mostrando cómo la

sistematización del control de existencias permite reducir la dependencia de registros manuales y mejorar la disponibilidad de la información para la toma de decisiones.

En la Universidad Católica (2017) se desarrolló un sistema web para el control de inventario del área de soporte técnico de Arturo Calle S.A.S., un antecedente relevante en cuanto al uso de aplicaciones web para registrar entradas, salidas y responsables de cada movimiento.

Estos antecedentes evidencian problemas comunes en los pequeños negocios: desabastecimiento, falta de trazabilidad de entradas y salidas y decisiones tomadas de manera empírica. A diferencia de ellos, el presente proyecto integra en un mismo sistema el inventario, las ventas y el cuadre de caja diario por método de pago, que es el punto crítico identificado en MundoCoco.

1.5 Justificación

Importancia para la organización. El desarrollo de un sistema web de gestión de inventarios para MundoCoco responde a una necesidad crítica identificada en sus procesos actuales. El registro manual, al estar a cargo de la persona de turno, representa una carga adicional en la jornada laboral, genera retrasos y una alta probabilidad de inconsistencias. Las discrepancias recurrentes en el cuadre de caja reflejan una falta de control sistemático sobre los ingresos que, acumulada en el tiempo, puede representar pérdidas económicas considerables y disminuir la confianza entre el equipo de trabajo y los propietarios.

Utilidad y aplicabilidad. El proyecto aborda problemas reales como los quiebres de stock, la acumulación de productos de baja rotación y la falta de trazabilidad en las entradas y salidas de mercancía. La solución genera reportes financieros y operativos automatizados, alertas de inventario en tiempo real y la conciliación de pagos por los diferentes canales que utiliza la tienda,

lo que mejora la eficiencia operativa y la calidad de la información disponible para la administración.

Factibilidad. El proyecto es viable en el marco académico y operativo: el equipo cuenta con formación en desarrollo web, bases de datos y metodologías ágiles; las herramientas utilizadas (PHP, Laravel, Filament, MySQL) son de código abierto y no generan costos de licenciamiento; y la administración de la tienda autorizó formalmente el desarrollo del proyecto, comprometiéndose a suministrar la información necesaria y a permitir las pruebas en su entorno real (ver Anexo 3).

En síntesis, este proyecto no solo resuelve una problemática operativa urgente, sino que también representa una oportunidad para reducir riesgos, mejorar la rentabilidad y alinear a MundoCoco con las tendencias de transformación digital de las pequeñas y medianas empresas.

2. Objetivos

2.1 Objetivo General

Desarrollar un sistema web de gestión de inventarios para MundoCoco que agilice los procesos de control de stock, registro de ventas y cuadro de caja, incorporando la categorización de métodos de pago, con el fin de mejorar la precisión operativa, reducir pérdidas y optimizar la rentabilidad del negocio.

2.2 Objetivos Específicos

1. Realizar el levantamiento de requisitos con el cliente MundoCoco mediante entrevistas y análisis de procesos, identificando necesidades críticas como la reducción de errores en registros, la disponibilidad de información en tiempo real, el control de pérdidas, la conciliación financiera diaria y la categorización de métodos de pago, para definir las funcionalidades del sistema de gestión de inventarios con trazabilidad completa.
2. Diseñar la arquitectura funcional del sistema web, definiendo los módulos de gestión de inventario, ventas y caja, acorde con los requisitos operativos de MundoCoco.
3. Desarrollar funcionalidades integradas para el cuadro de caja y el registro de ventas que permitan la conciliación financiera diaria, la actualización en tiempo real y la categorización de métodos de pago.
4. Validar el desempeño del sistema mediante pruebas de usabilidad y simulaciones de operación real en MundoCoco, midiendo los indicadores de precisión de inventario, tiempo de proceso y reducción de pérdidas.

3. Marco Referencial

En este capítulo se presentan los fundamentos teóricos, conceptuales, contextuales y legales que sustentan el desarrollo del proyecto, así como las tecnologías seleccionadas para su implementación.

3.1 Marco Teórico

3.1.1 Gestión de Inventarios

La gestión de inventarios es un proceso esencial para garantizar el equilibrio entre la oferta y la demanda, optimizando el flujo de productos y reduciendo pérdidas económicas. Una administración eficiente del inventario permite tomar decisiones estratégicas sobre compras, logística y disponibilidad (Waters, 2003). Según Ballou (2004), una gestión adecuada del inventario mejora el servicio al cliente, reduce los costos operativos y optimiza el flujo de materiales; en el contexto de las pequeñas empresas, la automatización de este proceso representa una oportunidad para ganar eficiencia sin requerir grandes inversiones.

Los sistemas modernos permiten automatizar tareas críticas como las alertas de escasez, la trazabilidad de movimientos y el análisis de los productos más vendidos (Bowersox et al., 2007). Para MundoCoco, que maneja productos perecederos como helados y bebidas, el control del stock mínimo y el registro de mermas son especialmente relevantes.

3.1.2 Control Financiero y Cuadre de Caja

El control financiero implica el seguimiento y análisis de los ingresos y egresos de una organización para garantizar su sostenibilidad económica. Gitman y Zutter (2012) señalan que un buen control financiero requiere información precisa, oportuna y confiable, lo que puede lograrse mediante sistemas automatizados que registren cada transacción en el momento en que ocurre. Las

discrepancias en el cuadro de caja, como las presentadas en MundoCoco, suelen ser indicio de fallas en los procesos o de la ausencia de herramientas adecuadas.

3.1.3 Sistemas de Información

Los sistemas de información son conjuntos integrados de componentes que recolectan, procesan, almacenan y distribuyen información para apoyar la toma de decisiones, el control y la coordinación en una organización (Laudon y Laudon, 2020). Sus componentes principales son el hardware, el software, los datos, los procedimientos y las personas. Los sistemas bien diseñados transforman procesos manuales en flujos automatizados, mejorando la precisión, la velocidad y la trazabilidad de la información.

3.1.4 Ingeniería de Requisitos

El levantamiento de requisitos constituye una etapa crítica de la ingeniería de software, ya que define lo que el sistema debe hacer y cómo debe comportarse. Según Sommerville (2011), una correcta recolección de requisitos mejora la calidad del producto final y reduce los costos derivados de errores posteriores. El uso de entrevistas, análisis de documentos y plantillas estructuradas permite obtener requisitos claros, completos y verificables (Pressman y Maxim, 2020).

3.1.5 Metodologías Ágiles y Scrum

Las metodologías ágiles surgieron como respuesta a la rigidez de los modelos tradicionales, priorizando la colaboración con el cliente, la entrega continua de software funcional y la respuesta al cambio (Beck et al., 2001). Scrum es un marco de trabajo ágil en el que el trabajo se organiza en ciclos cortos de duración fija llamados sprints, de entre una y cuatro semanas, al final de los cuales se obtiene un incremento del producto potencialmente utilizable (Schwaber y Sutherland, 2020).

Scrum define tres responsabilidades: el *Product Owner*, que representa las necesidades del negocio y prioriza el *Product Backlog*; el *Scrum Master*, que facilita el proceso y elimina impedimentos; y los *Developers*, que construyen el incremento. Sus eventos son la planificación del sprint (*Sprint Planning*), la reunión diaria (*Daily Scrum*), la revisión del sprint (*Sprint Review*) y la retrospectiva (*Sprint Retrospective*).

3.1.6 Arquitectura por Capas y Patrón MVC

La arquitectura por capas organiza una aplicación en niveles horizontales con responsabilidades separadas: presentación, lógica de negocio y datos. Esta separación facilita la mantenibilidad, la escalabilidad y las pruebas, porque cada capa puede evolucionar sin afectar a las demás (Sommerville, 2011). Laravel implementa el patrón Modelo-Vista-Controlador (MVC), en el que los modelos representan los datos, las vistas la interfaz y los controladores coordinan las solicitudes; en este proyecto se añadió una capa de servicios que concentra las reglas de negocio transaccionales.

3.1.7 PHP

PHP (*PHP: Hypertext Preprocessor*) es un lenguaje de programación de código abierto y de propósito general, especialmente adecuado para el desarrollo web del lado del servidor. El servidor interpreta el código PHP para generar contenido dinámico que luego se envía al navegador (The PHP Group, s.f.). El proyecto utiliza PHP 8.3, que incorpora tipado estricto, enumeraciones y mejoras de rendimiento.

3.1.8 Laravel

Laravel es un framework web de código abierto escrito en PHP, basado en el patrón MVC, que ofrece componentes listos para usar como autenticación, enrutamiento, sesiones, colas, un

ORM (Eloquent), migraciones de base de datos y un sistema de pruebas integrado (Laravel, s.f.). El proyecto utiliza Laravel 13.

3.1.9 Filament, Livewire y Tailwind CSS

Filament es un conjunto de componentes para Laravel que permite construir paneles administrativos completos (formularios, tablas, filtros, acciones, notificaciones y widgets) a partir de código PHP declarativo (Filament, s.f.). Se apoya en Livewire, que permite construir interfaces reactivas sin escribir JavaScript, y en Tailwind CSS, un framework de estilos basado en clases utilitarias. El proyecto utiliza Filament 5 con Tailwind CSS 4, compilado con Vite.

3.1.10 Sistema Gestor de Bases de Datos MySQL

MySQL es un sistema gestor de bases de datos relacional de código abierto, ampliamente utilizado en aplicaciones web por su rendimiento, confiabilidad y soporte transaccional mediante el motor InnoDB, que garantiza las propiedades ACID (atomicidad, consistencia, aislamiento y durabilidad) (Oracle, s.f.). El modelo relacional, propuesto por Codd (1970), organiza los datos en tablas relacionadas mediante claves, lo que facilita su integridad y consulta.

3.1.11 Control de Acceso Basado en Roles

El control de acceso basado en roles (RBAC) asigna permisos a roles y roles a usuarios, de modo que cada persona solo puede ejecutar las acciones propias de su función. En el proyecto se implementó con el paquete spatie/laravel-permission (Spatie, s.f.) y con las *policies* de Laravel, que consultan permisos granulares y no nombres de rol.

3.1.12 Pruebas de Software

Las pruebas de software garantizan que un sistema funcione como se espera bajo diferentes escenarios. Según Kaner et al. (1999), probar no solo implica encontrar errores, sino también validar que se cumplan los requisitos del cliente. Myers et al. (2011) proponen niveles de prueba

(unitarias, de integración, de sistema y de aceptación) que se aplicaron en este proyecto. Para la automatización se utilizó Pest, un framework de pruebas para PHP construido sobre PHPUnit, con una sintaxis expresiva orientada a describir comportamientos (Pest, s.f.). La usabilidad se evaluó con las diez heurísticas de Nielsen (1994).

3.1.13 Control de Versiones e Integración Continua

Git es un sistema de control de versiones distribuido que registra la historia de cambios del código fuente. GitHub Actions permite ejecutar automáticamente, en cada envío de código, un flujo de verificación (integración continua). En el proyecto, cada cambio ejecuta la revisión de estilo con Laravel Pint, la suite completa de pruebas, la medición de cobertura del código crítico y la compilación de los recursos del frontend.

3.2 Marco Conceptual

A continuación se definen los conceptos necesarios para comprender el dominio del sistema.

Inventario: conjunto de bienes o productos que una empresa tiene para su venta o producción. Su control permite determinar la disponibilidad de existencias y su valor económico.

Stock: cantidad física de un producto disponible para la venta. En el sistema corresponde al campo *stock actual* de cada producto.

Stock mínimo: cantidad por debajo de la cual se considera que un producto debe reabastecerse; al alcanzarla, el sistema genera una alerta y una sugerencia de reorden.

Inventario inicial: cantidad base registrada para un producto a partir de la cual se calcula el stock:

$$\text{Stock total} = \text{Inventario inicial} + \text{Entradas} - \text{Salidas}.$$

Movimiento de inventario (kardex): registro de cada entrada o salida de un producto con su fecha, tipo, cantidad, usuario responsable y stock antes y después del cambio.

Merma: pérdida de producto por vencimiento, daño u otra causa distinta de la venta.

Caja: registro diario del dinero de la tienda, compuesto por el saldo inicial (base), las ventas por método de pago y los gastos.

Cuadre de caja: proceso mediante el cual se verifica que el dinero físico disponible coincida con los registros de ingresos y egresos del día.

Saldo teórico: valor que debería haber en caja: $\text{Base} + \text{Ventas} - \text{Gastos}$.

Diferencia de caja: resultado de restar el saldo teórico al dinero contado; si es negativa hay faltante y si es positiva, sobrante.

Método de pago: canal por el cual el cliente paga una venta; en MundoCoco, efectivo, Nequi, transferencia bancaria o tarjeta.

Anulación de venta: reversión de una venta (devolución) que repone el inventario y la excluye de la caja y de los reportes sin borrarla.

Auditoría: registro histórico de las operaciones críticas realizadas en el sistema, con el usuario, la fecha y los valores anteriores y nuevos.

Rol y permiso: un permiso autoriza una acción concreta (por ejemplo, *crear ventas*); un rol agrupa permisos y se asigna a los usuarios.

Transacción: conjunto de operaciones sobre la base de datos que se ejecutan completas o no se ejecutan, evitando registros a medias.

3.3 Marco Contextual

MundoCoco es una tienda ubicada en la ciudad de Santiago de Cali, Valle del Cauca, dedicada a la venta de postres, helados, bebidas y productos derivados del coco. La tienda es administrada por la señora Jazmín Karina Meneses, quien autorizó formalmente el desarrollo del proyecto académico (ver Anexo 3). Su operación diaria está a cargo del personal de turno, que

atiende a los clientes, registra las ventas y, al final de la jornada, realiza el conteo del dinero y del inventario.

La tienda organiza su catálogo en cuatro líneas de producto, que fueron identificadas durante el levantamiento de información y se configuraron en el sistema con sus nombres exactos (Tabla 1).

Tabla 1. *Líneas de producto de MundoCoco*

Categoría	Productos
Helados	Arequipe, Bocadillo, Brownie, Café, Chicle, Chocolate, Fresa, Guanábana, Mangoviche, Mora, Maní, Maracuyá, Ron con Pasas, Salpicón
Bebidas	Limonada, Mandarinada, Cerezada, Avena, Cocofee, Cocoloco, Cocobaileys, Jugo de Coco, Limonada Natural, Agua de Coco, Agua Natural, Agua con Gas, Bolis
Aceites	Presentaciones de 30 ml, 60 ml, 100 ml, 130 ml, 200 ml, 500 ml, 1 litro y cosmético de 120 ml
Productos de Coco	Leche de Coco, PET 500/1000, Galón, Tortas, Cocadas, Hojuelas, Galletas

Nota. Elaboración propia a partir del levantamiento de requisitos (documento RF y RNF).

Los actores que intervienen en los procesos de la tienda y su correspondencia con los roles del sistema se describen en la Tabla 2.

Tabla 2. *Actores de MundoCoco y su rol en el sistema*

Actor	Funciones en la tienda	Rol en el sistema
Administradora	Abrir y cerrar la caja, registrar compras y gastos, supervisar el inventario, gestionar el personal.	Administrador (acceso total)
Personal de turno	Atender clientes, registrar ventas y consultar la disponibilidad de productos.	Operario
Propietario	Revisar los resultados del negocio, las ventas y el flujo de caja.	Consultor (solo reportes)

Nota. Elaboración propia.

Antes del proyecto, el flujo de información era el siguiente: el personal de turno anotaba las ventas en una hoja física, al cierre sumaba los valores y comparaba el resultado con el dinero en caja en el reverso de la misma hoja, y entregaba los registros al propietario para su

procesamiento contable. Este flujo no distinguía los pagos por método, no registraba los gastos de materia prima y no permitía conocer el stock real sin un conteo físico.

3.4 Marco Legal

El desarrollo del sistema se enmarca en la normatividad colombiana relacionada con los derechos de autor, la propiedad intelectual del software, la protección de datos personales y la validez de los mensajes de datos, así como en estándares internacionales de ingeniería de software.

3.4.1 Ley 23 de 1982 – Derechos de Autor

Protege las obras literarias, científicas y artísticas, incluidos los programas de computador como creaciones intelectuales (Congreso de Colombia, 1982). Los autores del sistema son titulares de los derechos morales sobre el código fuente, el diseño y la documentación del aplicativo.

3.4.2 Ley 603 de 2000 – Propiedad Intelectual en Software

Exige que las empresas informen sobre el cumplimiento de las normas de propiedad intelectual del software que utilizan (Congreso de Colombia, 2000). Todas las dependencias del proyecto (Laravel, Filament, Sanctum, DomPDF, PhpSpreadsheet, entre otras) se distribuyen con licencia MIT u otras licencias libres y están declaradas en el archivo *composer.json*.

3.4.3 Ley 1581 de 2012 y Decreto 1377 de 2013 – Protección de Datos Personales

Establecen el derecho de las personas a conocer, actualizar, rectificar y suprimir los datos personales recolectados en bases de datos, y exigen la autorización previa e informada del titular para su tratamiento (Congreso de Colombia, 2012; Presidencia de la República, 2013). En cumplimiento de esta norma, el sistema publica una política de tratamiento de datos, exige que cada usuario la acepte antes de operar y registra la fecha de aceptación como prueba.

3.4.4 Ley 1266 de 2008 – Habeas Data

Aunque se enfoca en la información financiera y crediticia, su principio de manejo responsable de datos se aplica de forma complementaria a los registros de ventas, gastos y caja del sistema, que solo son visibles según el rol del usuario (Congreso de Colombia, 2008).

3.4.5 Ley 527 de 1999 – Comercio Electrónico y Mensajes de Datos

Reconoce la validez jurídica de los mensajes de datos (Congreso de Colombia, 1999). Los registros y reportes generados electrónicamente por el sistema, con su fecha de generación, tienen un valor probatorio equivalente al de los documentos físicos que reemplazan.

3.4.6 Estándares de Ingeniería de Software

ISO/IEC/IEEE 29148:2018. Define los procesos y el contenido para la especificación de requisitos de sistemas y software. Sirvió de guía para redactar los requisitos funcionales y no funcionales de forma clara, verificable y trazable (ISO, 2018).

ISO/IEC/IEEE 12207:2017. Establece los procesos del ciclo de vida del software, desde la concepción hasta el mantenimiento. Se utilizó como marco de referencia para las fases de planificación, diseño, construcción, verificación y transición del proyecto (ISO, 2017).

ISO/IEC/IEEE 29119. Conjunto de normas que definen los procesos, la documentación y las técnicas de prueba de software. Orientó el plan de pruebas funcionales, unitarias y de integración (ISO, 2022).

ISO/IEC 25010. Define el modelo de calidad del producto software (adecuación funcional, eficiencia de desempeño, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad, entre otras características), que se utilizó como referencia para agrupar los requisitos no funcionales (ISO, 2023).

4. Método

El proyecto se desarrolló bajo la metodología ágil Scrum, tal como se estableció en el anteproyecto aprobado. Se eligió por su enfoque iterativo, su flexibilidad ante cambios en los requisitos y la posibilidad de validar con el cliente, al final de cada ciclo, un incremento funcional del sistema. Este enfoque resultó adecuado para un equipo de dos integrantes y para un cliente con disponibilidad limitada, que necesitaba ver avances concretos para retroalimentar el desarrollo.

El ciclo de vida del software se organizó en las fases propuestas por la norma ISO/IEC/IEEE 12207 —planificación, diseño, codificación y pruebas—, ejecutadas de manera incremental dentro de los sprints. La arquitectura del sistema sigue un modelo por capas sobre el patrón Modelo-Vista-Controlador de Laravel, con una capa de servicios que concentra las reglas de negocio.

4.1 Metodología de Desarrollo

4.1.1 Roles de Scrum

Los roles de Scrum se asignaron de la siguiente manera (Tabla 3). Dado el tamaño del equipo, los dos integrantes compartieron las responsabilidades de desarrollo, pruebas y documentación, y coordinaron la comunicación con la tienda.

Tabla 3. Roles de Scrum en el proyecto

Rol	Responsable	Responsabilidades
Product Owner	Jazmín Karina Meneses, administradora de MundoCoco	Describir las necesidades del negocio, priorizar el Product Backlog y validar los incrementos en cada revisión.
Scrum Master	Cristian Camilo Díaz Ramírez	Facilitar los eventos de Scrum, hacer seguimiento al tablero de tareas y eliminar impedimentos.
Developers	Cristian Camilo Díaz Ramírez y Carlos Daniel Camacho Valencia	Análisis, diseño, codificación, pruebas automatizadas y documentación del sistema.
Asesor académico	James Alexis Angulo Medina	Orientar metodológicamente el proyecto y revisar los entregables académicos.

Nota. Elaboración propia.

4.1.2 Artefactos y Eventos

El *Product Backlog* se construyó a partir del documento de requisitos funcionales y no funcionales, expresando cada necesidad como una historia de usuario priorizada (Tabla 7). En cada *Sprint Planning* se seleccionaron las historias de mayor prioridad y se descompusieron en tareas; durante el sprint, el avance se revisó en reuniones cortas y el código se integró en el repositorio Git del proyecto, alojado en GitHub. En cada *Sprint Review* se presentó el incremento a la administración de la tienda para recibir observaciones, y en la retrospectiva se identificaron mejoras para el ciclo siguiente.

Como práctica de calidad, cada cambio enviado al repositorio ejecutó automáticamente el flujo de integración continua definido en GitHub Actions: revisión de estilo de código, ejecución de toda la suite de pruebas, medición de cobertura del código crítico y compilación de los recursos del frontend. Un cambio solo se consideraba terminado (*Definition of Done*) cuando el flujo completo finalizaba sin errores.

4.1.3 Planificación de Sprints

El cronograma del anteproyecto contempló 18 semanas: 2 para el levantamiento de requisitos, 2 para el diseño, 8 para el desarrollo, 2 para las pruebas internas y correcciones y 4 para las pruebas con el cliente. La Tabla 4 muestra la distribución de ese trabajo en sprints y los incrementos entregados en cada uno.

Tabla 4. *Planificación de sprints e incrementos entregados*

Sprint	Duración	Objetivo	Incremento entregado
0	2 semanas	Levantamiento de requisitos	Entrevistas con la administración, análisis de los formatos manuales, documento de requisitos funcionales y no funcionales, Product Backlog priorizado.
1	2 semanas	Diseño del sistema	Arquitectura, modelo entidad-relación, diagramas UML, prototipos de interfaz y configuración del repositorio.

Sprint	Duración	Objetivo	Incremento entregado
2	2 semanas	Base del catálogo y ventas	Migraciones y modelos; módulos de categorías, productos y métodos de pago; formulario de ventas con descuento automático de stock; módulo de caja con totales por método de pago.
3	2 semanas	Operación diaria	Escritorio con indicadores, datos iniciales de MundoCoco (categorías y productos), validación de stock, cierre de caja con diferencia, interfaz en español.
4	2 semanas	Seguridad y calidad	Usuarios, roles y permisos con políticas por módulo; estrategia de pruebas automatizadas con Pest.
5	2 semanas	Trazabilidad y reportes	Movimientos de inventario (RF02, RF03, RF12), gastos en la caja, auditoría, reportes con exportación PDF/Excel/CSV, respaldo diario de la base de datos.
6	2 semanas	Cierre de cumplimiento	Servicio único de ventas, anulación de ventas, Nequi como método de pago, precio aplicado auditado, consentimiento Ley 1581, indicadores de gestión, sucursales y API de integración.
7	4 semanas	Pruebas con el cliente	Pruebas internas y correcciones, pruebas de usabilidad con el personal, capacitación, manual de usuario y manual técnico.

Nota. Elaboración propia.

4.2 Fase de Planificación

En esta fase se recopiló la información necesaria para definir el alcance del sistema: se realizaron entrevistas con la administración de la tienda, se analizaron los formatos manuales utilizados para el inventario y la caja (hojas de registro y archivos de Excel) y se observó el proceso de cierre de la jornada. Con esta información se especificaron los requisitos funcionales y no funcionales siguiendo las recomendaciones de la norma ISO/IEC/IEEE 29148.

4.2.1 Levantamiento de Información

Las entrevistas se realizaron con la administradora de la tienda y con el personal encargado de la venta y del cierre de caja, utilizando la guía de preguntas del Anexo 1. [COMPLETAR: número de entrevistas realizadas, personas entrevistadas y periodo en que se realizaron]. Adicionalmente se revisaron los formatos físicos y el archivo de Excel con el que la tienda llevaba el control de caja, a partir del cual se identificaron los campos de la base de caja, las ventas en efectivo y por Nequi y los gastos del día.

Del levantamiento se concluyó que la tienda no contaba con un registro confiable del stock, que la persona de turno debía extender su jornada para completar el inventario, que el cuadre de caja presentaba diferencias frecuentes sin posibilidad de rastrear su origen y que no se registraban los gastos de materia prima. Se solicitaron como funcionalidades prioritarias el registro de ventas con descuento automático del inventario, el cuadre de caja separado por método de pago, las alertas de productos por agotarse, los reportes de ventas e inventario exportables y el control de usuarios por roles.

Durante el análisis se identificaron diferencias entre los documentos iniciales del proyecto que se resolvieron con el cliente: el documento de requisitos mencionaba como métodos de pago únicamente *Efectivo* y *Nequi*, mientras que el anteproyecto mencionaba *efectivo, transferencias bancarias y tarjetas*; se decidió soportar los cuatro métodos y mostrar Nequi por separado en la caja. Asimismo, el requisito RF04 permite aplicar un precio distinto al precio base; se implementó de forma controlada, validando en el servidor que el precio esté dentro del rango permitido y registrando en la auditoría cada diferencia.

4.2.2 Requisitos Funcionales

Tabla 5. Requisitos funcionales

ID	Nombre	Descripción	Entrada	Salida	Prioridad
RF01	Gestión de productos	Crear, consultar, actualizar y eliminar productos del catálogo, con código único autogenerado.	Nombre, categoría, precio de venta (\$2.000 a \$95.000), precio de costo, stock mínimo, descripción.	Producto registrado y disponible para inventario y ventas.	Alta
RF02	Inventario inicial	Registrar el inventario inicial de cada producto.	Producto, cantidad inicial, fecha de registro.	Stock base establecido y movimiento tipo inicial.	Alta
RF03	Registro de entradas	Registrar entradas de mercancía: compras, devoluciones y ajustes positivos, de uno o varios productos.	Productos, cantidades (> 0), fecha y hora, tipo, observaciones.	Stock actualizado y movimientos en el historial.	Alta

ID	Nombre	Descripción	Entrada	Salida	Prioridad
RF04	Procesamiento de ventas	Registrar ventas y actualizar automáticamente el inventario; impedir ventas sin stock suficiente y alertar cuando el stock quede bajo el mínimo.	Productos, cantidades, precio aplicado, fecha y hora, método de pago.	Venta registrada, total calculado, stock descontado, alerta de mínimo.	Alta
RF05	Cálculo de stock total	Calcular el stock como Inventario inicial + Entradas – Salidas.	Movimientos registrados.	Stock calculado en tiempo real y detección de inconsistencias.	Alta
RF06	Gestión de categorías	Organizar los productos en Helados, Bebidas, Aceites y Productos de Coco.	Nombre y descripción de la categoría.	Categoría disponible para clasificar productos.	Media
RF07	Alertas de stock mínimo	Generar alertas cuando un producto esté en su stock mínimo o por debajo.	Stock mínimo configurado por producto.	Notificación en el escritorio, lista de productos críticos y sugerencia de reorden.	Media
RF08	Reportes de inventario	Generar reportes de inventario por categoría, stock bajo, valorización y movimientos por periodo.	Tipo de reporte y rango de fechas.	Reporte en pantalla y exportable a PDF, Excel y CSV.	Media
RF09	Reportes de ventas	Generar reportes de ventas diarias por producto, por categoría, más vendidos, ingresos por periodo y comparativo.	Tipo de reporte y rango de fechas.	Reporte en pantalla y exportable.	Media
RF10	Usuarios y roles	Manejar los roles Administrador (acceso total), Operario (ventas y consulta de inventario) y Consultor (solo reportes); crear usuarios, asignar roles y cambiar contraseñas.	Nombre, correo, contraseña, roles.	Usuario con acceso restringido según su rol.	Media
RF11	Control de caja diaria	Integrar el control de caja: base, ventas por método de pago y gastos, con cuadre automático.	Saldo inicial, ventas, gastos, dinero contado.	Saldo teórico = Base + Ventas – Gastos; diferencia al cierre; flujo de efectivo diario.	Baja
RF12	Trazabilidad de movimientos	Mantener el historial completo de movimientos de inventario con búsqueda por producto, fecha, usuario y tipo.	Movimientos generados por el sistema.	Historial con fecha y hora, usuario, tipo, cantidad y stock antes y después.	Baja

Nota. Elaboración propia a partir del documento de requisitos validado con MundoCoco.

4.2.3 Requisitos No Funcionales

Tabla 6. *Requisitos no funcionales*

ID	Nombre	Descripción
RNF01	Rendimiento	Tiempo de respuesta máximo de 3 segundos por operación; soporte mínimo para 200 productos, 2.000 movimientos mensuales y 5 usuarios simultáneos.

ID	Nombre	Descripción
RNF02	Usabilidad	Interfaz intuitiva y completamente en español, para usuarios sin experiencia técnica; máximo 2 horas de capacitación para el uso básico; diseño adaptable a tabletas y escritorio.
RNF03	Compatibilidad	Navegadores modernos (Chrome, Firefox, Safari, Edge); resoluciones desde 1024×768 hasta 1920×1080; sin instalación de software adicional.
RNF04	Seguridad	Inicio de sesión obligatorio; contraseñas cifradas con bcrypt; cierre de sesión tras 2 horas de inactividad; auditoría de operaciones críticas; respaldo automático diario.
RNF05	Confiabilidad	Transacciones ACID en operaciones críticas; validación en cliente y servidor; mensajes de error descriptivos.
RNF06	Mantenibilidad	Código documentado y organizado con patrones estándar; parámetros configurables sin modificar código; manual técnico.
RNF07	Escalabilidad	Arquitectura que permita agregar módulos, preparada para múltiples sucursales y para la integración con sistemas externos.
RNF08	Portabilidad	Accesible desde cualquier dispositivo con navegador; compatible con MySQL, PostgreSQL o SQL Server.
RNF09	Eficiencia	Páginas principales con carga inferior a 2 segundos; base de datos inicial menor a 100 MB.
RNF10	Cumplimiento	Cumplimiento de la legislación colombiana de protección de datos, estándares web y cobertura mínima de pruebas del 80 % en las funcionalidades críticas.

Nota. Elaboración propia.

4.2.4 Product Backlog

Los requisitos se tradujeron en historias de usuario con el formato *Como [rol], quiero [funcionalidad], para [beneficio]*. La Tabla 7 presenta el Product Backlog priorizado y el sprint en que se entregó cada historia.

Tabla 7. *Product Backlog: historias de usuario*

ID	Como...	Quiero...	Para...	Req.	Prioridad	Sprint
HU-01	Administrador	registrar productos con su categoría, precio y stock mínimo	mantener el catálogo actualizado	RF01, RF06	Alta	2
HU-02	Operario	registrar una venta con varios productos y su método de pago	que el stock se descuente sin cálculos manuales	RF04	Alta	2
HU-03	Administrador	abrir y cerrar la caja del día con el dinero contado	conocer al instante si hay faltante o sobrante	RF11	Alta	2-3
HU-04	Administrador	ver en el escritorio los productos en stock mínimo	reabastecer a tiempo	RF07	Media	3
HU-05	Administrador	crear usuarios y asignarles un rol	que cada persona solo acceda a lo que le corresponde	RF10	Media	4

ID	Como...	Quiero...	Para...	Req.	Prioridad	Sprint
HU-06	Administrador	registrar el inventario inicial y las compras de varios productos	que el stock refleje la mercancía real	RF02, RF03	Alta	5
HU-07	Administrador	registrar mermas y ajustes negativos con su motivo	controlar las pérdidas	RF03, RF12	Media	5
HU-08	Administrador	registrar los gastos de materia prima y servicios	que la caja los descuente del saldo	RF11	Media	5
HU-09	Administrador	consultar el historial de movimientos filtrado por producto, fecha, usuario o tipo	rastrear cualquier diferencia de inventario	RF12	Baja	5
HU-10	Consultor	descargar reportes de ventas e inventario en PDF o Excel	analizar el negocio	RF08, RF09	Media	5
HU-11	Administrador	anular una venta indicando el motivo	registrar devoluciones sin borrar información	RF04, RF12	Media	6
HU-12	Operario	cambiar el precio de una línea dentro del rango permitido	aplicar descuentos autorizados	RF04	Media	6
HU-13	Administrador	ver la caja separada en efectivo, Nequi, transferencias y tarjetas	conciliar cada canal de pago	RF11	Alta	6
HU-14	Usuario	aceptar la política de tratamiento de datos al ingresar	cumplir la Ley 1581 de 2012	RNF10	Media	6
HU-15	Administrador	consultar indicadores de precisión de inventario y pérdidas	medir el impacto del sistema	OE4	Media	6
HU-16	Administrador	contar con un respaldo diario automático de la base de datos	no perder información	RNF04	Alta	5

Nota. Elaboración propia.

4.3 Fase de Diseño

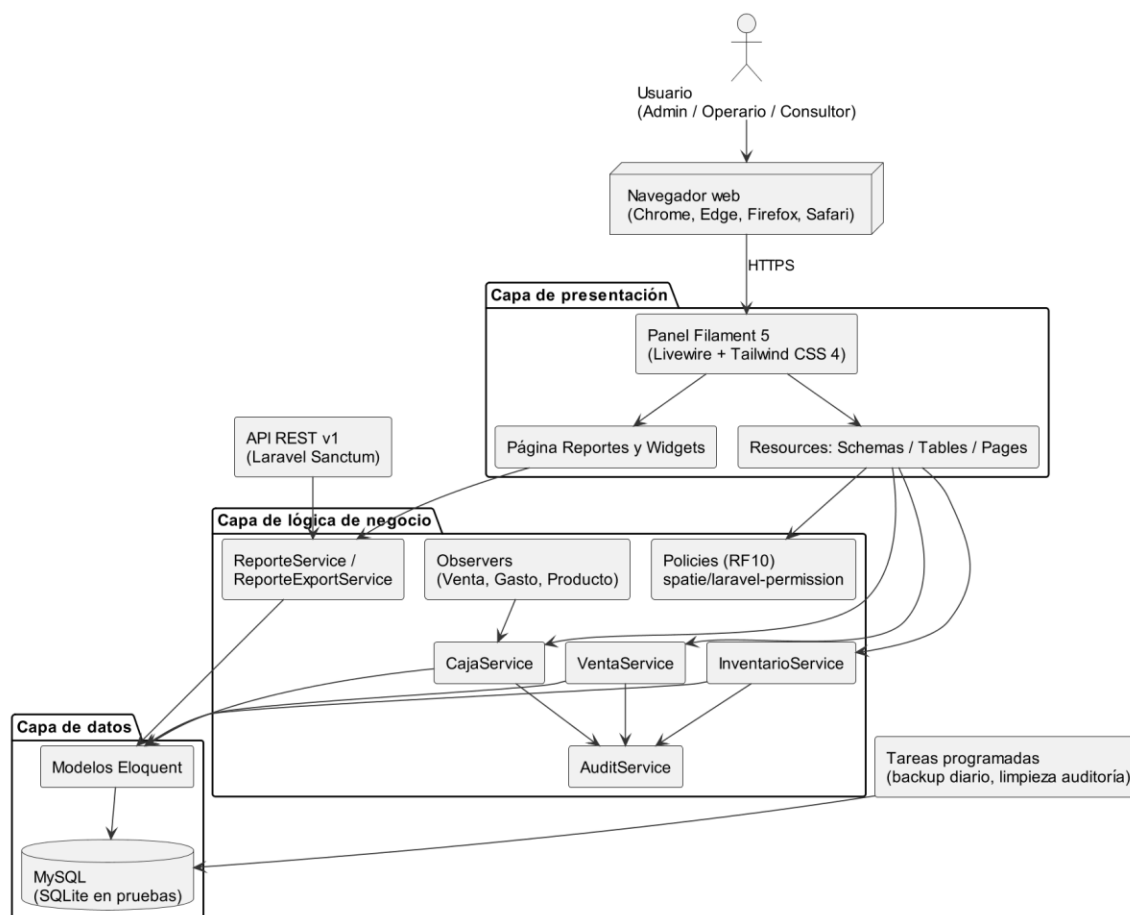
En esta fase se definió la estructura del sistema a partir de los requisitos: la arquitectura, el modelo de datos y los modelos UML que describen el comportamiento de los principales procesos (casos de uso, secuencia y actividades). También se definió la organización de las interfaces del panel administrativo.

4.3.1 Arquitectura del Sistema

El sistema sigue una arquitectura por capas (Figura 1). La **capa de presentación** está construida con Filament 5: cada módulo se define como un *Resource* con su formulario (*Schema*), su tabla (*Table*) y sus páginas de listado, creación y edición; las pantallas no contienen reglas de negocio, solo delegan en los servicios. La **capa de lógica de negocio** está formada por los servicios

VentaService, *CajaService*, *InventarioService*, *ReporteService*, *ReporteExportService* y *AuditService*, por las *policies* que controlan los permisos y por los *observers* que recalculan la caja abierta cuando cambian las ventas o los gastos. La **capa de datos** está compuesta por los modelos Eloquent y la base de datos MySQL. Adicionalmente, el sistema ofrece una API REST de solo lectura para integraciones externas y tareas programadas para el respaldo diario y la limpieza de la auditoría.

Figura 1. *Arquitectura por capas del sistema*

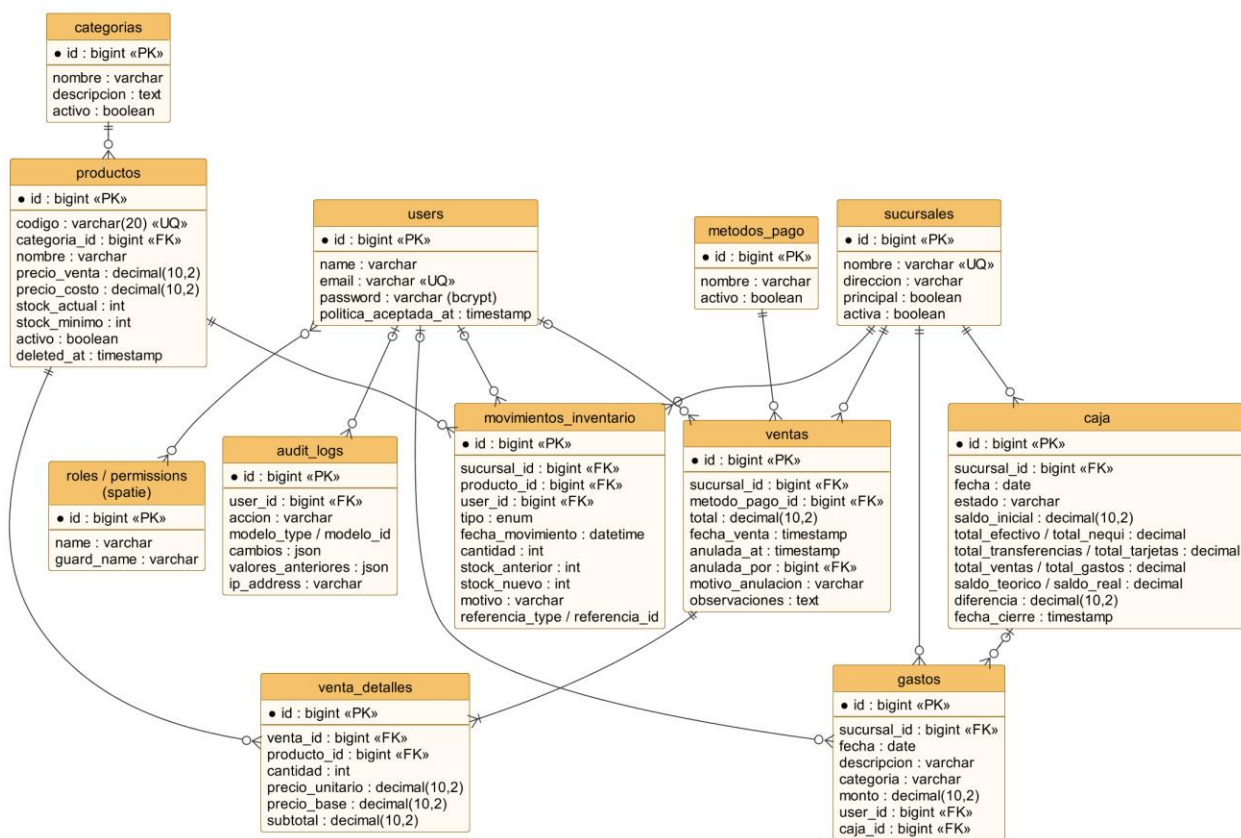


Nota. Elaboración propia.

4.3.2 Modelo Entidad-Relación

El modelo entidad-relación representa las entidades del dominio, sus atributos y las relaciones entre ellas (Chen, 1976). La Figura 2 presenta el modelo implementado en las migraciones del proyecto.

Figura 2. Diagrama entidad-relación



Nota. Elaboración propia.

Las relaciones principales son las siguientes:

- Una **categoría** agrupa muchos **productos**; cada producto pertenece a una sola categoría.
- Una **venta** tiene uno o más **detalles**; cada detalle referencia un **producto** y guarda la cantidad, el precio base, el precio aplicado y el subtotal.

- Un **método de pago** puede aparecer en muchas ventas; cada venta tiene un único método de pago.
- Cada **movimiento de inventario** corresponde a un producto y al usuario que lo registró, y puede referenciar la venta que lo originó (relación polimórfica *referencia*).
- Cada **gasto** se asocia a una fecha, al usuario que lo registró y, opcionalmente, a la **caja** de esa fecha.
- Las **ventas**, **cajas**, **gastos** y **movimientos** pertenecen a una **sucursal**; por defecto, la sucursal principal.
- Los **usuarios** tienen **roles** y los roles agrupan **permisos** (tablas del paquete spatie/laravel-permission).

4.3.3 Diccionario de Datos

La base de datos se definió mediante migraciones de Laravel, lo que permite recrearla de forma idéntica en cualquier motor compatible. Los valores monetarios se almacenan como *decimal(10,2)* y se operan internamente en centavos enteros para evitar errores de redondeo. A continuación se describen las tablas principales del dominio.

Tabla 8. Diccionario de datos, tabla productos

Campo	Tipo	Descripción
id	bigint (PK)	Identificador del producto.
codigo	varchar(20), único	Código del producto; si se deja vacío se genera automáticamente (P-000001).
categoria_id	bigint (FK)	Categoría a la que pertenece.
nombre / descripcion	varchar / text	Nombre y descripción del producto.
precio_venta	decimal(10,2)	Precio base de venta (entre \$2.000 y \$95.000, configurable).
precio_costo	decimal(10,2)	Costo del producto, usado en la valorización del inventario.
stock_actual / stock_minimo	int	Existencias actuales y umbral de alerta.
activo	boolean	Indica si el producto puede venderse.
deleted_at	timestamp	Borrado lógico: el producto sale del catálogo sin perder su historial.

Nota. Elaboración propia.

Tabla 9. *Diccionario de datos, tabla ventas y venta_detalle*

Campo	Tipo	Descripción
ventas.metodo_pago_id	bigint (FK)	Método de pago de la venta.
ventas.total	decimal(10,2)	Total calculado en el servidor.
ventas.fecha_venta	timestamp	Fecha y hora de la venta; determina la caja a la que pertenece.
ventas.anulada_at / anulada_por / motivo_anulacion	timestamp / FK / varchar	Datos de la anulación (devolución).
venta_detalle.producto_id	bigint (FK)	Producto vendido.
venta_detalle.cantidad	int	Unidades vendidas.
venta_detalle.precio_base	decimal(10,2)	Precio de catálogo al momento de la venta.
venta_detalle.precio_unitario / subtotal	decimal(10,2)	Precio aplicado y subtotal de la línea.

Nota. Elaboración propia.

Tabla 10. *Diccionario de datos, tabla caja*

Campo	Tipo	Descripción
fecha / estado	date / varchar	Día de la caja y estado (abierta o cerrada).
saldo_inicial	decimal(10,2)	Base con la que se abre la caja.
total_efectivo / total_nequi / total_transferencias / total_tarjetas	decimal(10,2)	Ventas del día por método de pago.
total_ventas / total_gastos	decimal(10,2)	Totales de ventas y gastos del día.
saldo_teorico	decimal(10,2)	Base + ventas – gastos.
saldo_real / diferencia	decimal(10,2)	Dinero contado al cierre y su diferencia con el saldo teórico.
fecha_cierre / observaciones_cierre	timestamp / text	Momento y observaciones del cierre.

Nota. Elaboración propia.

Tabla 11. *Diccionario de datos, tabla movimientos_inventario*

Campo	Tipo	Descripción
producto_id / user_id	bigint (FK)	Producto afectado y usuario que registró el movimiento.
tipo	enum	inicial, compra, devolucion, ajuste_positivo, venta, ajuste_negativo o merma.

Campo	Tipo	Descripción
fecha_movimiento	datetime	Fecha de la transacción (created_at guarda el momento exacto del registro).
cantidad	int	Unidades del movimiento; el tipo determina si suma o resta.
stock_anterior / stock_nuevo	int	Stock antes y después del movimiento.
motivo / observaciones	varchar / text	Justificación del movimiento.
referencia_type / referencia_id	polimórfica	Registro que originó el movimiento (por ejemplo, una venta).

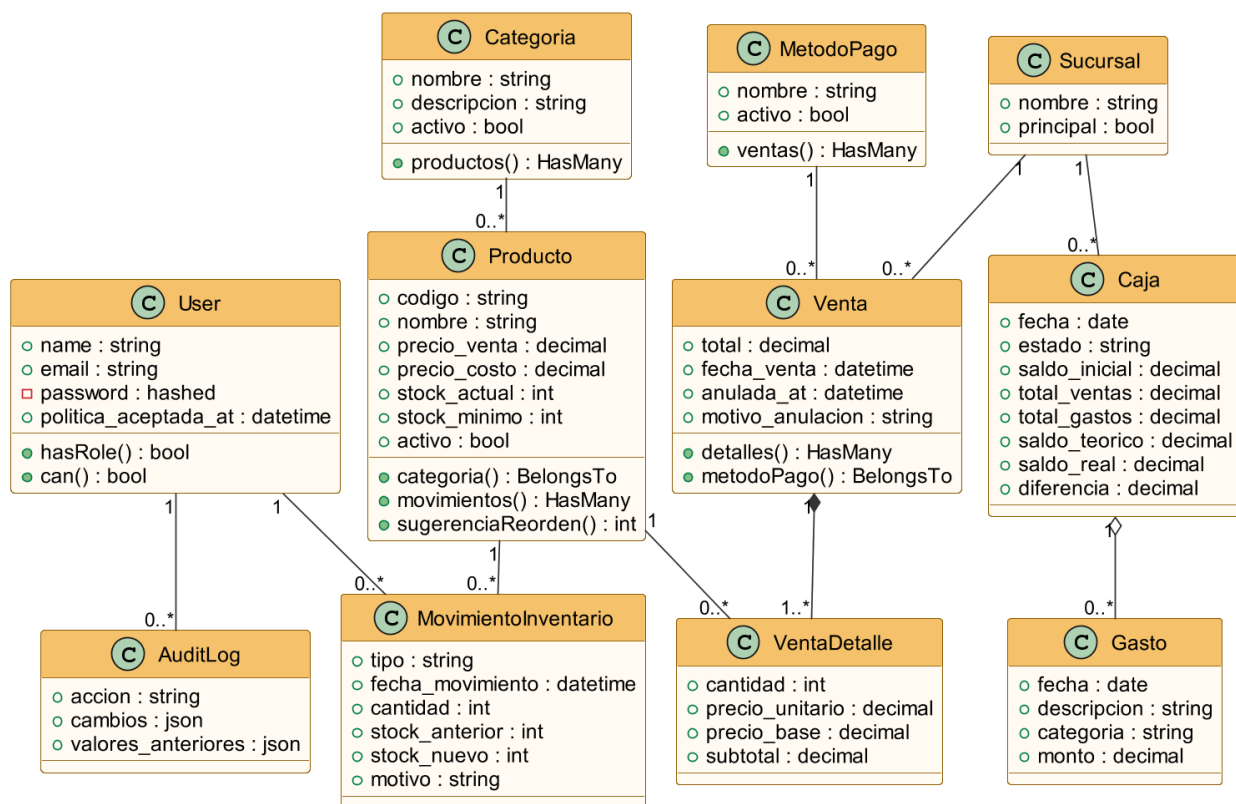
Nota. Elaboración propia.

Las demás tablas del sistema son *categorias*, *metodos_pago*, *gastos* (fecha, descripción, categoría del gasto, monto, usuario y caja), *audit_logs* (usuario, acción, registro afectado, cambios, valores anteriores, dirección IP y navegador), *sucursales*, *users* (incluye la fecha de aceptación de la política de datos), las tablas de roles y permisos, y *personal_access_tokens* para la API.

4.3.4 Diagrama de Clases

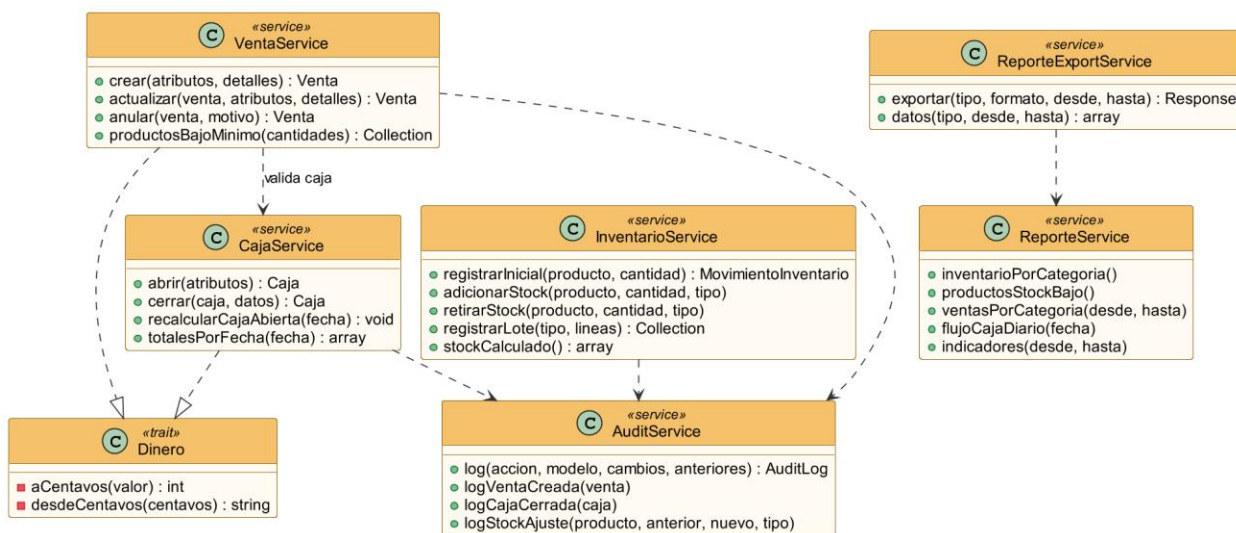
Un diagrama de clases representa la estructura estática del sistema: sus clases, atributos, métodos y relaciones (Booch et al., 1999). La Figura 3 muestra los modelos del dominio y la Figura 4 los servicios que implementan las reglas de negocio.

Figura 3. Diagrama de clases del dominio



Nota. Elaboración propia.

Figura 4. Diagrama de clases de la capa de servicios

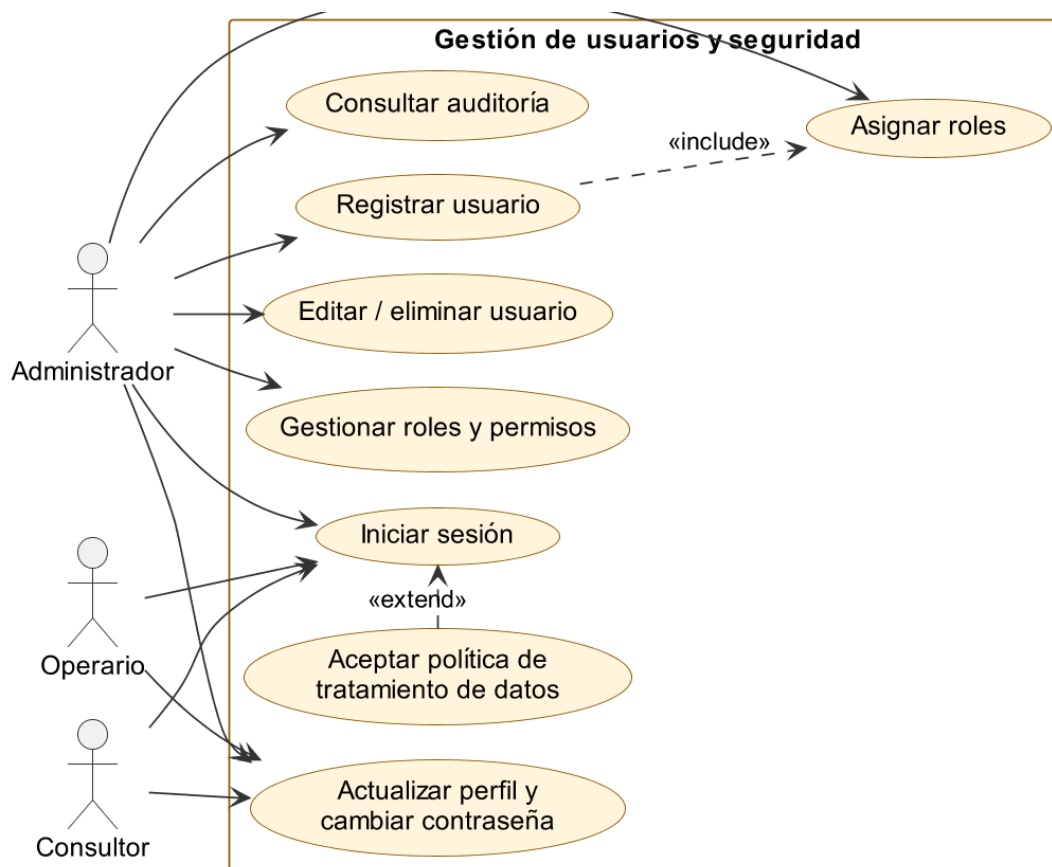


Nota. Elaboración propia.

4.3.5 Diagramas de Casos de Uso

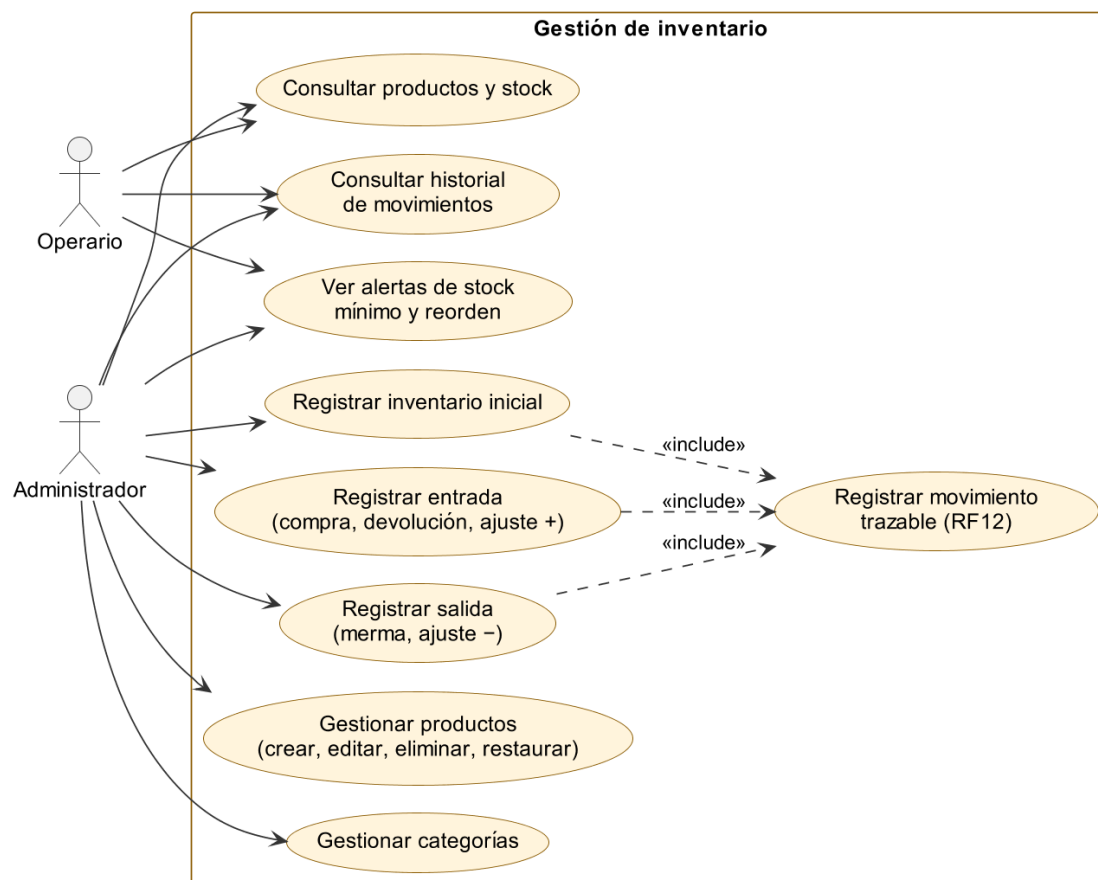
Los diagramas de casos de uso describen las interacciones entre los actores y las funcionalidades del sistema. Se identificaron tres actores, que corresponden a los roles del sistema: Administrador, Operario y Consultor. Las Figuras 5 a 8 presentan los casos de uso agrupados por módulo.

Figura 5. Diagrama de casos de uso, gestión de usuarios y seguridad



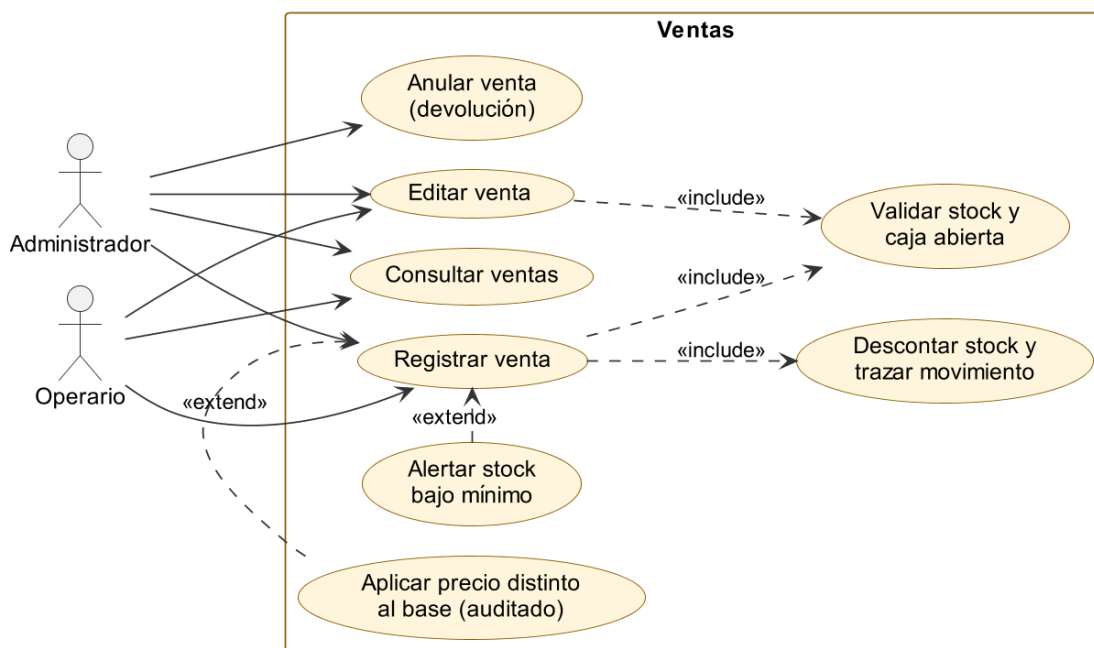
Nota. Elaboración propia.

Figura 6. Diagrama de casos de uso, gestión de inventario



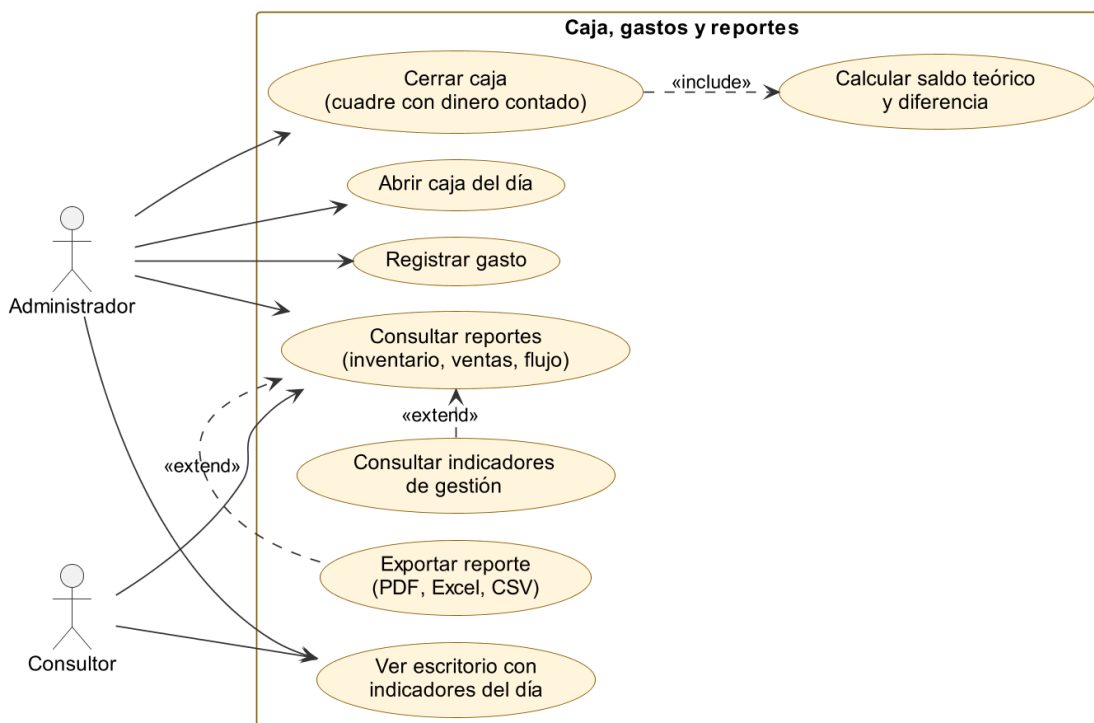
Nota. Elaboración propia.

Figura 7. Diagrama de casos de uso, ventas



Nota. Elaboración propia.

Figura 8. Diagrama de casos de uso, caja, gastos y reportes



Nota. Elaboración propia.

4.3.6 Casos de Uso Extendidos

A continuación se detallan los casos de uso principales en formato extendido, con su curso normal de eventos y sus eventos alternos.

Tabla 12. Caso de uso extendido CU-01, Iniciar sesión

Identificador: CU-01	Título: Iniciar sesión
INFORMACIÓN GENERAL	
ACTORES	Administrador, Operario, Consultor
PROPÓSITO	Permitir el acceso autenticado al sistema.
RESUMEN	El usuario ingresa su correo y contraseña; el sistema valida las credenciales, verifica que tenga un rol y que haya aceptado la política de datos, y muestra el escritorio según sus permisos.
REQUISITOS	RF10, RNF04, Ley 1581
PRECONDICIONES	El usuario está registrado y tiene al menos un rol.
POSTCONDICIONES	Se crea una sesión que expira tras 2 horas de inactividad.

CURSO NORMAL DE LOS EVENTOS	
Acciones de los actores	Respuestas del sistema
1. Ingresar a /admin.	2. Muestra el formulario de ingreso con el enlace a la política de privacidad.
3. Escribe correo y contraseña y pulsa Entrar.	4. Valida el formato del correo y verifica la contraseña cifrada con bcrypt.
	5. Si el usuario no ha aceptado la política, lo redirige a la página de consentimiento.
6. Acepta la política (solo en el primer ingreso).	7. Registra la fecha de aceptación, lo audita y muestra el escritorio con los módulos de su rol.

EVENTOS ALTERNOS
Paso 4: si las credenciales no coinciden, muestra "Estas credenciales no coinciden con nuestros registros".
Paso 4: si un campo está vacío, indica que es obligatorio.
Paso 5: un usuario sin rol no puede entrar al panel.

Nota. Elaboración propia.

Tabla 13. Caso de uso extendido CU-02, Registrar usuario y asignar rol

Identificador: CU-02	Título: Registrar usuario y asignar rol
-----------------------------	--

INFORMACIÓN GENERAL	
ACTORES	Administrador
PROPÓSITO	Crear cuentas para el personal con el rol que le corresponde.
RESUMEN	El administrador registra nombre, correo, contraseña y roles de un nuevo usuario.
REQUISITOS	RF10
PRECONDICIONES	El administrador ha iniciado sesión.
POSTCONDICIONES	El nuevo usuario puede iniciar sesión con los permisos de su rol.

CURSO NORMAL DE LOS EVENTOS	
Acciones de los actores	Respuestas del sistema
1. Selecciona Seguridad → Usuarios → Nuevo usuario.	2. Muestra el formulario de usuario.
3. Diligencia nombre, correo, contraseña (mínimo 12 caracteres) y su confirmación, y selecciona los roles.	4. Valida los datos, cifra la contraseña y guarda el usuario con sus roles.
	5. Muestra la notificación de creación exitosa.

EVENTOS ALTERNOS
Paso 4: si el correo ya existe, indica que debe ser único.
Paso 4: si la contraseña no cumple la longitud o la confirmación no coincide, muestra el error.
Un Operario o Consultor no ve la opción Usuarios (acceso prohibido).

Nota. Elaboración propia.

Tabla 14. *Caso de uso extendido CU-03, Registrar producto*

Identificador: CU-03	Título: Registrar producto
INFORMACIÓN GENERAL	
ACTORES	Administrador
PROPÓSITO	Agregar un producto al catálogo.
RESUMEN	El administrador registra un producto con categoría, precios, stock y stock mínimo; si indica stock, el sistema registra el inventario inicial.
REQUISITOS	RF01, RF02, RF06
PRECONDICIONES	Existe al menos una categoría activa.
POSTCONDICIONES	El producto queda disponible para inventario y ventas con código único.
CURSO NORMAL DE LOS EVENTOS	
Acciones de los actores	Respuestas del sistema

1. Selecciona Catálogo → Productos → Nuevo producto.	2. Muestra el formulario con la ayuda del rango de precio permitido.
3. Diligencia categoría, nombre, descripción, precio de venta, precio de costo, stock y stock mínimo.	4. Valida los datos, genera el código si está vacío y guarda el producto.
	5. Si el stock es mayor que cero, registra un movimiento de tipo inventario inicial.

EVENTOS ALTERNOS

Paso 4: si el precio está fuera del rango \$2.000 – \$95.000, muestra el error.

Paso 4: si el código indicado ya existe, informa que debe ser único.

Nota. Elaboración propia.

Tabla 15. Caso de uso extendido CU-04, Registrar movimiento de inventario

Identificador: CU-04	Título: Registrar movimiento de inventario
INFORMACIÓN GENERAL	
ACTORES	Administrador
PROPÓSITO	Registrar entradas y salidas de mercancía distintas de las ventas.
RESUMEN	El administrador registra una compra, devolución, ajuste o merma de uno o varios productos en una sola operación.
REQUISITOS	RF02, RF03, RF05, RF12
PRECONDICIONES	Los productos existen en el catálogo.
POSTCONDICIONES	El stock se actualiza y cada línea queda en el historial con stock anterior y nuevo.

CURSO NORMAL DE LOS EVENTOS

Acciones de los actores	Respuestas del sistema
1. Selecciona Catálogo → Movimientos → Nuevo movimiento.	2. Muestra el formulario con las líneas de productos.
3. Elige el tipo, la fecha y hora, el motivo y agrega las líneas (producto y cantidad).	4. Bloquea los productos, valida cantidades y stock y actualiza el inventario en una sola transacción.
	5. Registra un movimiento y una entrada de auditoría por línea y muestra la confirmación.

EVENTOS ALTERNOS

Paso 4: si una cantidad es cero o negativa, o la fecha es futura, muestra la validación.

Paso 4: si una salida supera el stock de algún producto, no se guarda ninguna línea del lote.

Nota. Elaboración propia.

Tabla 16. *Caso de uso extendido CU-05, Registrar venta*

Identificador: CU-05	Título: Registrar venta
INFORMACIÓN GENERAL	
ACTORES	Operario, Administrador
PROPÓSITO	Registrar la venta de uno o varios productos y descontar el inventario.
RESUMEN	El usuario selecciona el método de pago, la fecha y hora y los productos; el sistema calcula el total con los precios del servidor, descuenta el stock y actualiza la caja.
REQUISITOS	RF04, RF05, RF07, RF11, RF12
PRECONDICIONES	La caja de la fecha no está cerrada y los productos están activos.
POSTCONDICIONES	La venta queda registrada, el stock descontado, la caja recalculada y los movimientos trazados.

CURSO NORMAL DE LOS EVENTOS	
Acciones de los actores	Respuestas del sistema
1. Selecciona Operación diaria → Ventas → Nueva venta.	2. Muestra el formulario con la fecha actual y el precio base sugerido.
3. Elige el método de pago y la fecha/hora; agrega productos, cantidades y, si aplica, un precio distinto.	4. Calcula subtotales y total en pantalla.
5. Pulsa Crear.	6. Valida fecha, caja, stock (agrupando líneas repetidas) y precio; guarda la venta, descuenta el stock, registra movimientos y auditoría, recalcula la caja y muestra la confirmación con la alerta de stock mínimo si aplica.

EVENTOS ALTERNOS
Paso 6: si el stock no alcanza, muestra "Stock insuficiente para [producto]" y no guarda nada.
Paso 6: si la caja de la fecha está cerrada o la fecha es futura, rechaza la venta.
Paso 6: si el precio aplicado está fuera del rango permitido, muestra el error.

Nota. Elaboración propia.

Tabla 17. *Caso de uso extendido CU-06, Anular venta*

Identificador: CU-06	Título: Anular venta
INFORMACIÓN GENERAL	
ACTORES	Administrador
PROPÓSITO	Registrar una devolución sin eliminar la venta.
RESUMEN	El administrador anula una venta indicando el motivo; el sistema repone el inventario y la excluye de la caja y de los reportes.

REQUISITOS	RF04, RF11, RF12, RNF04
PRECONDICIONES	La venta no está anulada y la caja de su fecha está abierta.
POSTCONDICIONES	La venta queda marcada como anulada con su motivo; el stock repuesto queda como devolución.

CURSO NORMAL DE LOS EVENTOS	
Acciones de los actores	Respuestas del sistema
1. En el listado de ventas pulsa Anular.	2. Muestra el diálogo de confirmación.
3. Escribe el motivo y confirma.	4. Repone el stock (movimientos tipo devolución), marca la venta como anulada, recalcula la caja y registra la auditoría.

EVENTOS ALTERNOS
Paso 4: sin motivo, o con la caja cerrada, la anulación se rechaza.
Una venta anulada no puede editarse ni anularse de nuevo.

Nota. Elaboración propia.

Tabla 18. Caso de uso extendido CU-07, Abrir caja

Identificador: CU-07	Título: Abrir caja
INFORMACIÓN GENERAL	
ACTORES	Administrador
PROPÓSITO	Iniciar el control de caja del día.
RESUMEN	El administrador registra la fecha y el saldo inicial; el sistema calcula los totales de ventas y gastos ya registrados para esa fecha.
REQUISITOS	RF11
PRECONDICIONES	No existe otra caja abierta para la misma fecha.
POSTCONDICIONES	La caja queda abierta y se recalcula con cada venta o gasto.

CURSO NORMAL DE LOS EVENTOS	
Acciones de los actores	Respuestas del sistema
1. Selecciona Operación diaria → Cajas → Nueva caja.	2. Muestra el formulario de apertura.
3. Indica la fecha y el saldo inicial.	4. Calcula los totales por método de pago, los gastos y el saldo teórico, y guarda la caja abierta.

EVENTOS ALTERNOS
Paso 4: si ya existe una caja abierta para esa fecha, muestra "Ya existe una caja abierta para esta fecha".

Nota. Elaboración propia.

Tabla 19. *Caso de uso extendido CU-08, Registrar gasto*

Identificador: CU-08	Título: Registrar gasto
INFORMACIÓN GENERAL	
ACTORES	Administrador
PROPÓSITO	Registrar los egresos del día.
RESUMEN	El administrador registra un gasto de materia prima, servicios, transporte u otros, que se descuenta del saldo teórico de la caja de su fecha.
REQUISITOS	RF11
PRECONDICIONES	La caja de la fecha no está cerrada.
POSTCONDICIONES	El gasto queda registrado y la caja abierta se recalcula.

CURSO NORMAL DE LOS EVENTOS	
Acciones de los actores	Respuestas del sistema
1. Selecciona Operación diaria → Gastos → Nuevo gasto.	2. Muestra el formulario de gasto.
3. Indica fecha, descripción, categoría, monto y observaciones.	4. Guarda el gasto y recalcula el saldo teórico de la caja.

EVENTOS ALTERNOS
Paso 4: si la caja de la fecha está cerrada, no permite registrar, editar ni eliminar el gasto.

Nota. Elaboración propia.

Tabla 20. *Caso de uso extendido CU-09, Cerrar caja*

Identificador: CU-09	Título: Cerrar caja
INFORMACIÓN GENERAL	
ACTORES	Administrador
PROPÓSITO	Realizar el cuadro de caja al final de la jornada.
RESUMEN	El administrador ingresa el dinero contado; el sistema calcula el saldo teórico y la diferencia, y bloquea la fecha.
REQUISITOS	RF11, RNF04
PRECONDICIONES	La caja está abierta.
POSTCONDICIONES	La caja queda cerrada; no se pueden registrar ni modificar ventas o gastos de esa fecha.

CURSO NORMAL DE LOS EVENTOS	
Acciones de los actores	Respuestas del sistema
1. En el listado de cajas pulsa Cerrar caja.	2. Muestra el diálogo de cierre.
3. Escribe el dinero contado y las observaciones, y confirma.	4. Recalcula los totales por fecha de venta, calcula saldo teórico = base + ventas – gastos y diferencia = contado – saldo teórico, cierra la caja y registra la auditoría.

EVENTOS ALTERNOS
Paso 4: si la caja ya fue cerrada, informa "La caja ya fue cerrada".
Paso 4: si el valor contado no es un monto válido, muestra el error.

Nota. Elaboración propia.

Tabla 21. Caso de uso extendido CU-10, Consultar y exportar reportes

Identificador: CU-10	Título: Consultar y exportar reportes
INFORMACIÓN GENERAL	
ACTORES	Administrador, Consultor
PROPÓSITO	Analizar el inventario, las ventas y la caja.
RESUMEN	El usuario elige un periodo, consulta los reportes en pantalla y los descarga en PDF, Excel o CSV.
REQUISITOS	RF08, RF09, RF11, OE4
PRECONDICIONES	El usuario tiene el permiso ver reportes.
POSTCONDICIONES	El reporte se muestra o se descarga con la información del periodo.

CURSO NORMAL DE LOS EVENTOS	
Acciones de los actores	Respuestas del sistema
1. Selecciona Informes → Reportes.	2. Muestra los reportes del periodo por defecto.
3. Indica las fechas Desde/Hasta y pulsa Aplicar periodo.	4. Calcula valorización, indicadores, inventario por categoría, stock bajo, comparativo, movimientos, ventas y flujo de efectivo.
5. Pulsa PDF, Excel o CSV en la sección deseada.	6. Verifica el permiso exportar reportes y genera el archivo.

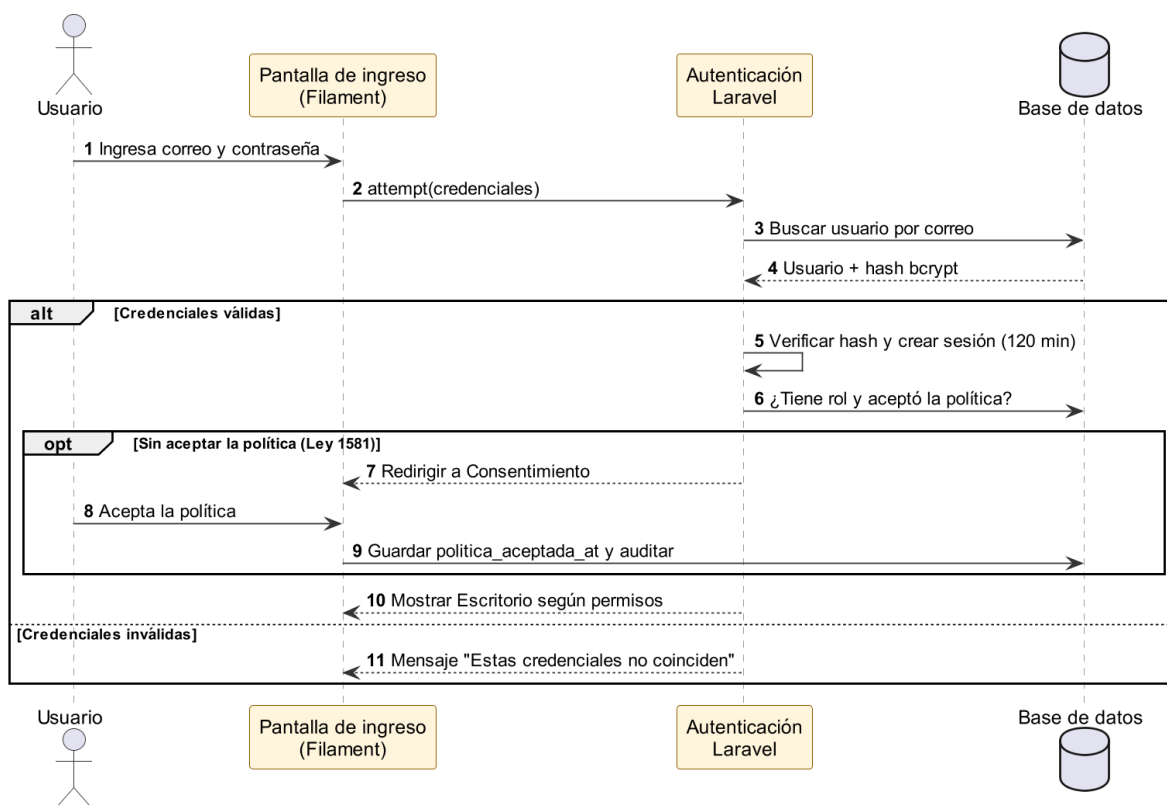
EVENTOS ALTERNOS
Paso 6: si el rango de fechas no es válido, responde con un error 422.
Un Operario no tiene acceso a los reportes.

Nota. Elaboración propia.

4.3.7 Diagramas de Secuencia

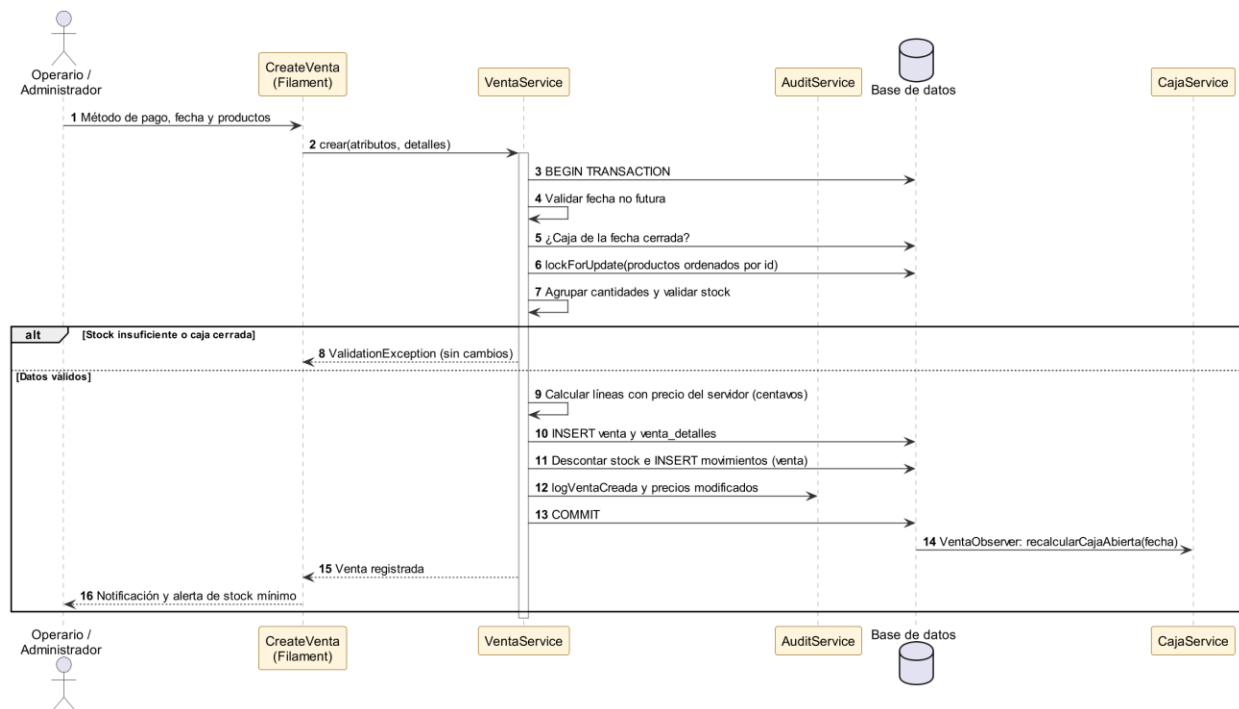
Los diagramas de secuencia representan la interacción entre los objetos del sistema en orden temporal (Fowler, 2004). Se elaboraron para los procesos críticos: inicio de sesión, registro de venta, cierre de caja, registro de movimientos de inventario y generación de reportes.

Figura 9. Diagrama de secuencia, iniciar sesión



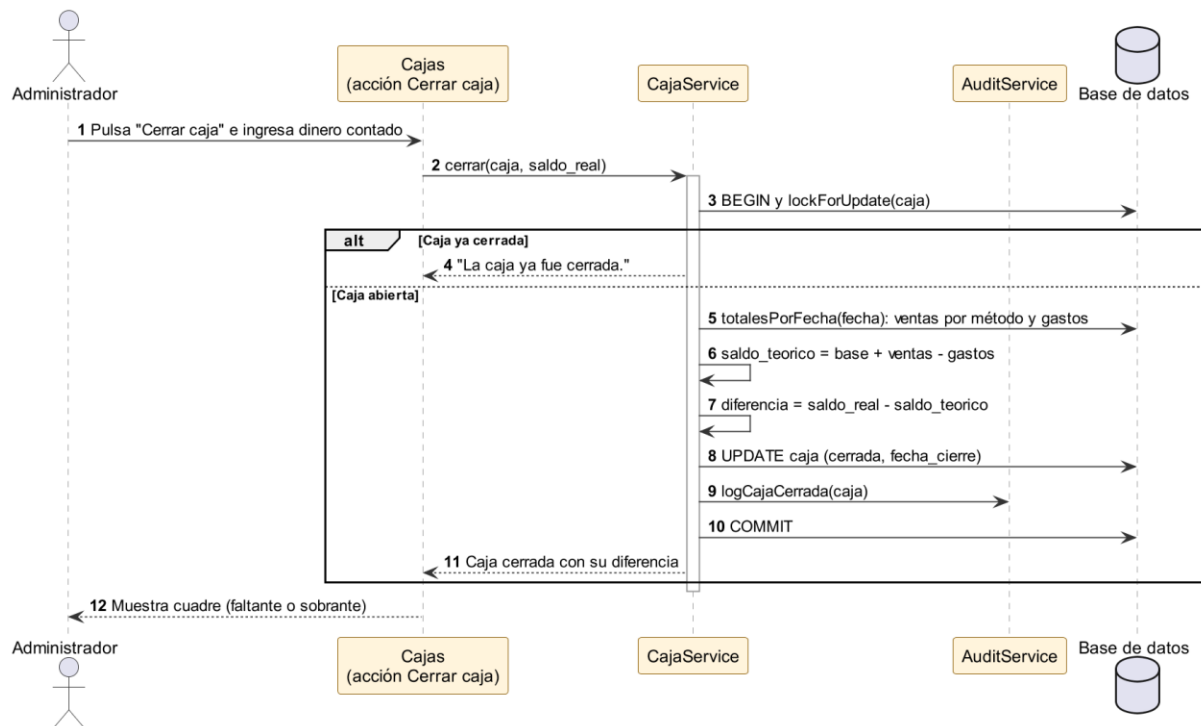
Nota. Elaboración propia.

Figura 10. Diagrama de secuencia, registrar venta



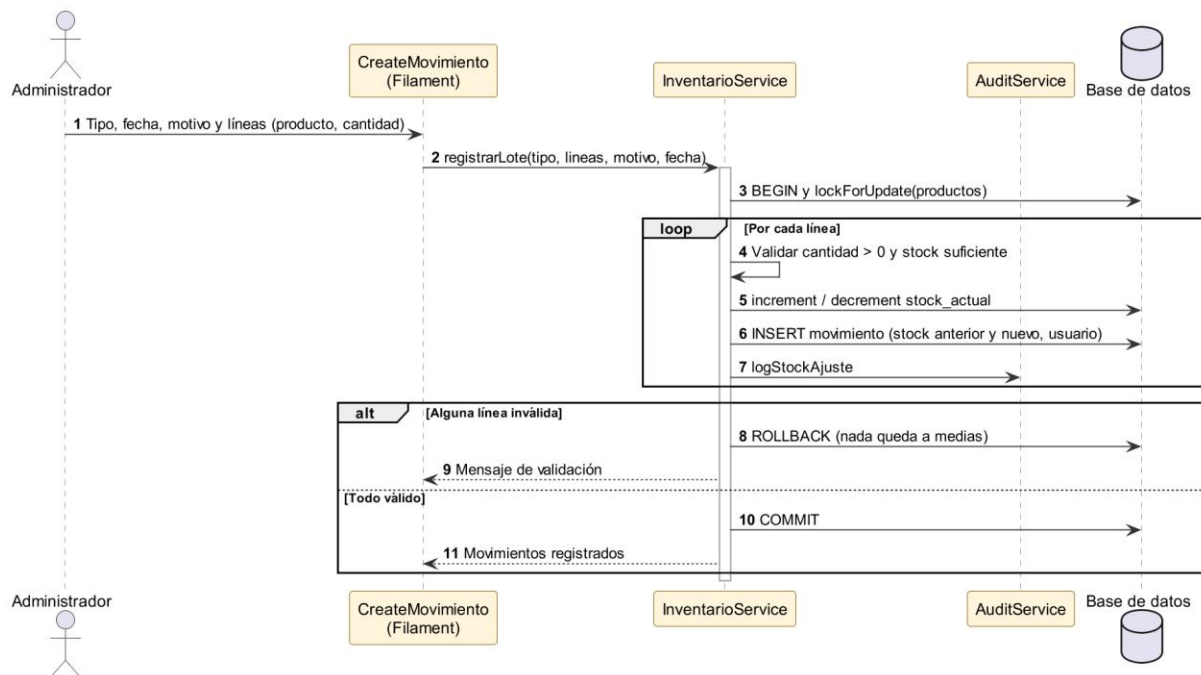
Nota. Elaboración propia.

Figura 11. Diagrama de secuencia, cerrar caja



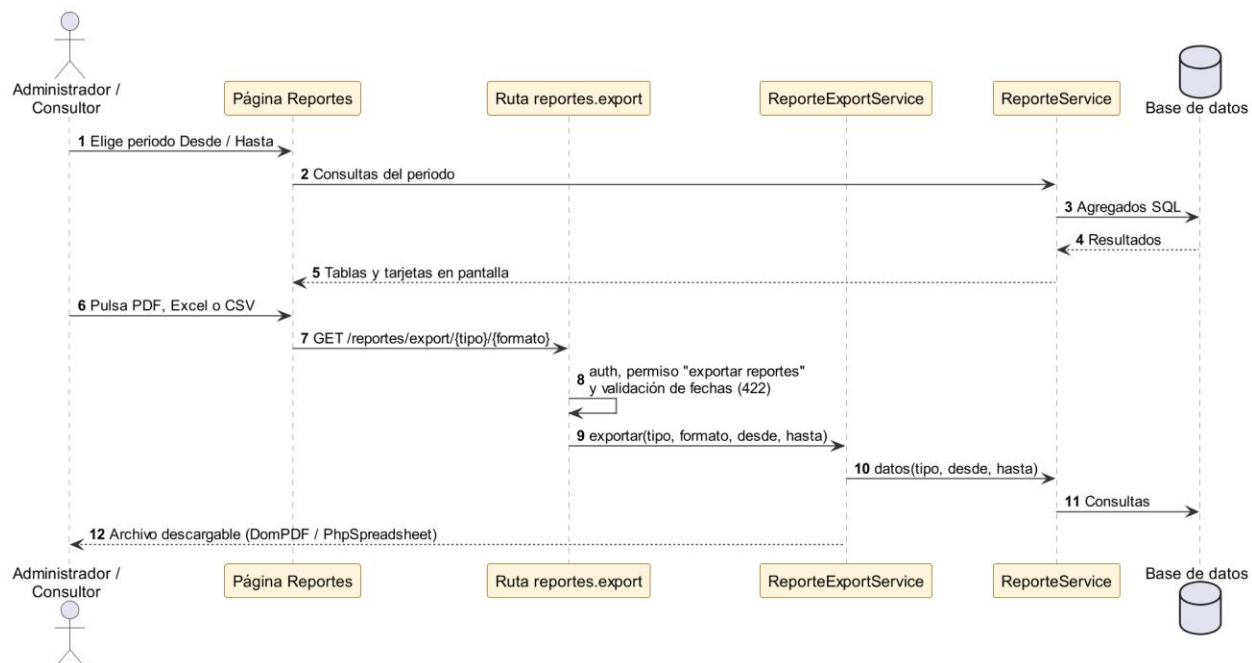
Nota. Elaboración propia.

Figura 12. Diagrama de secuencia, registrar movimiento de inventario



Nota. Elaboración propia.

Figura 13. Diagrama de secuencia, consultar y exportar reportes

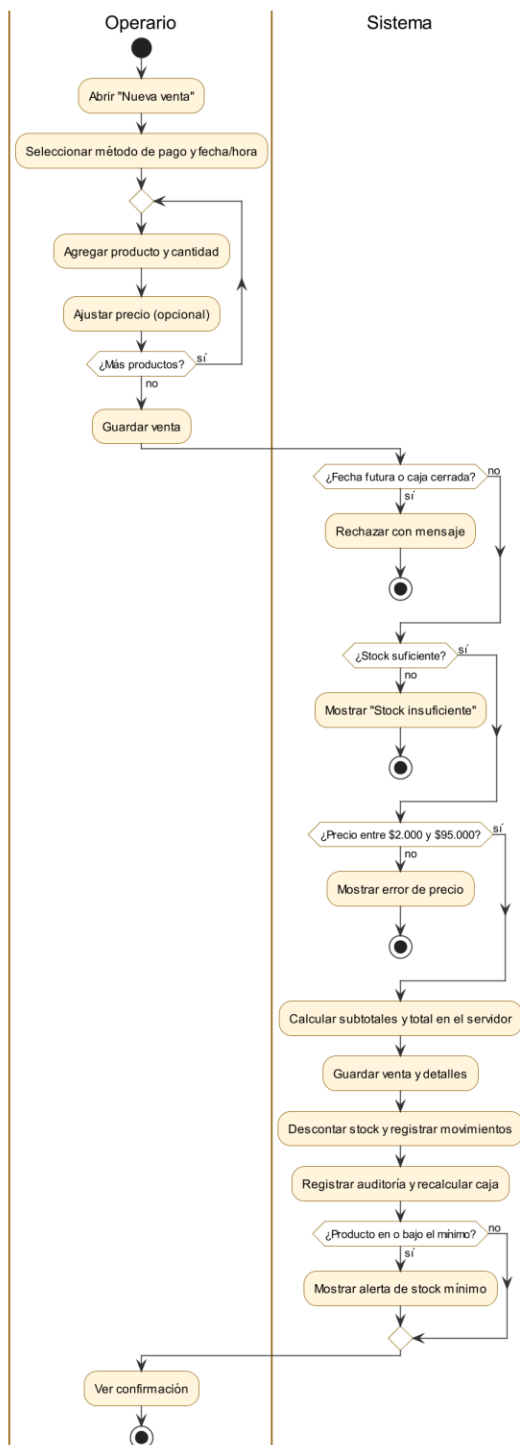


Nota. Elaboración propia.

4.3.8 Diagramas de Actividades

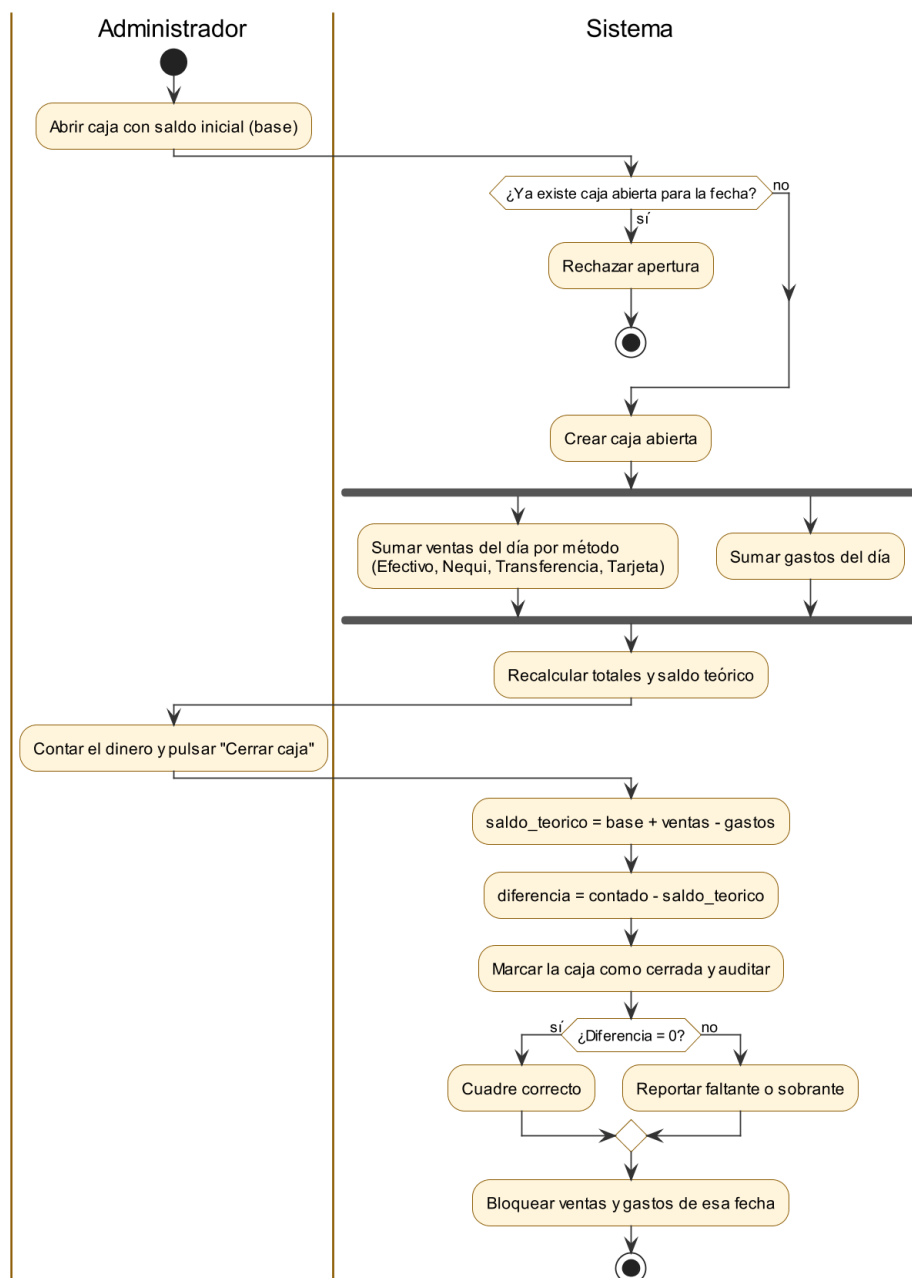
Los diagramas de actividades describen el flujo de trabajo de un proceso, con sus decisiones y resultados. Se elaboraron para los tres procesos operativos centrales de la tienda.

Figura 14. Diagrama de actividades, registrar venta



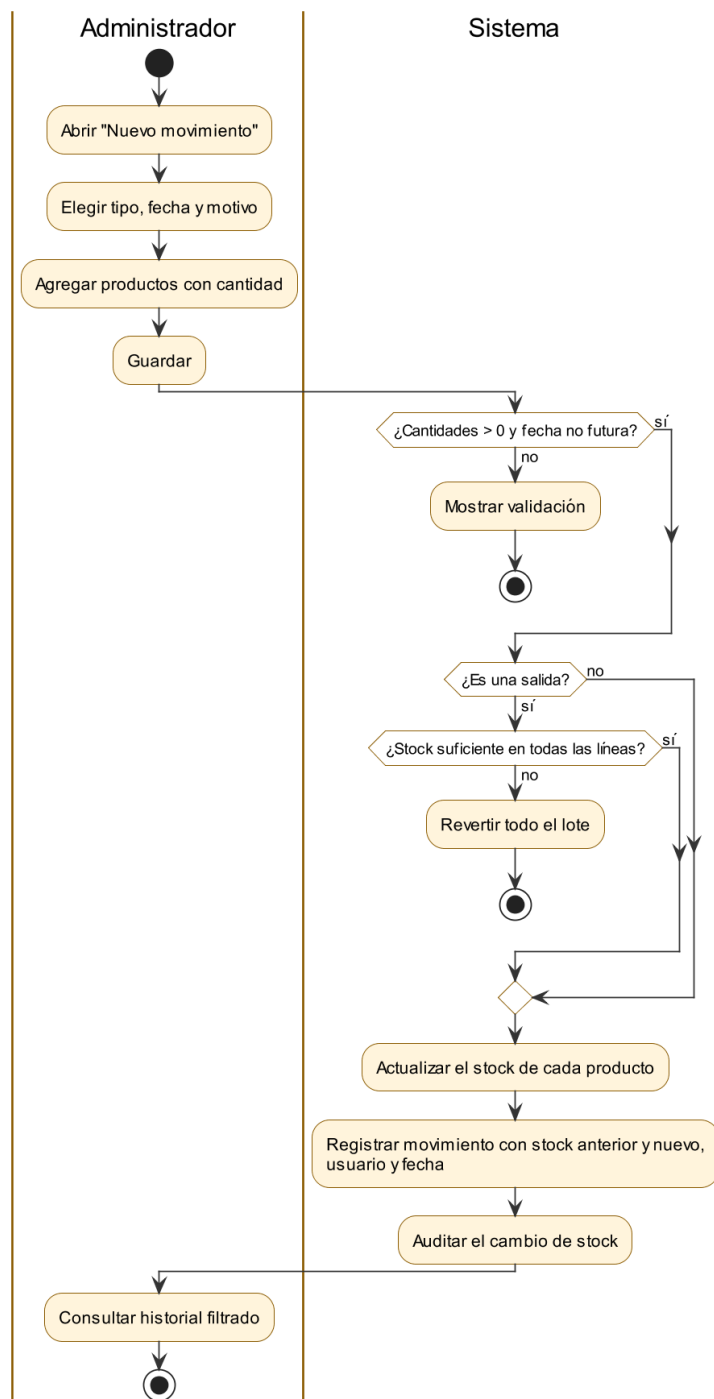
Nota. Elaboración propia.

Figura 15. Diagrama de actividades, apertura y cuadre de caja



Nota. Elaboración propia.

Figura 16. Diagrama de actividades, registrar movimiento de inventario



Nota. Elaboración propia.

4.3.9 Diseño de Interfaces

Las interfaces se diseñaron sobre el panel administrativo de Filament, que ofrece una estructura consistente para todos los módulos: un menú lateral agrupado (*Operación diaria*, *Catálogo*, *Informes* y *Seguridad*), listados con búsqueda, filtros y paginación, y formularios con validación en línea. El menú solo muestra los módulos a los que el usuario tiene permiso. Se priorizó la interfaz en español, el uso de colores para estados (por ejemplo, stock en rojo cuando está en el mínimo) y la confirmación de las acciones irreversibles. Las interfaces finales implementadas se presentan en la sección 4.4.

4.4 Fase de Codificación

Con los requisitos y el diseño aprobados se construyó el sistema de manera incremental, sprint a sprint. Esta sección describe el entorno de desarrollo, la organización del código y la implementación de cada módulo, acompañada de las interfaces finales y de fragmentos del código fuente.

4.4.1 Entorno y Herramientas de Desarrollo

Tabla 22. *Tecnologías y herramientas utilizadas*

Componente	Tecnología / versión	Uso en el proyecto
Lenguaje	PHP 8.3	Lógica del servidor.
Framework	Laravel 13	Enrutamiento, ORM Eloquent, migraciones, autenticación, colas, tareas programadas.
Panel administrativo	Filament 5 (Livewire)	Formularios, tablas, filtros, acciones, widgets y notificaciones.
Estilos y compilación	Tailwind CSS 4, Vite 8, Node 20	Estilos de la interfaz y compilación de recursos.
Base de datos	MySQL (producción), SQLite (pruebas)	Persistencia transaccional.
Roles y permisos	spatie/laravel-permission, filament-spatie-roles-permissions	Control de acceso basado en roles (RF10).
API	Laravel Sanctum	Autenticación por tokens para la API de integración (RNF07).

Componente	Tecnología / versión	Uso en el proyecto
Reportes	DomPDF, PhpSpreadsheet	Exportación de reportes a PDF y Excel (RF08, RF09).
Pruebas	Pest 4 (sobre PHPUnit)	Pruebas unitarias, de integración y de funcionalidad.
Estilo de código	Laravel Pint (PSR-12)	Verificación automática del formato del código.
Entorno local	Laragon, Visual Studio Code	Servidor local, base de datos y edición del código.
Control de versiones	Git, GitHub, GitHub Actions	Repositorio del proyecto e integración continua.

Nota. Elaboración propia.

4.4.2 Organización del Código

El código sigue la estructura estándar de Laravel, complementada con la organización de Filament 5. Los directorios más relevantes son:

- *app/Services/*: reglas de negocio transaccionales (ventas, caja, inventario, reportes, exportación y auditoría). Es el único lugar donde se modifica el stock o se calculan los totales.
- *app/Filament/Resources/*: un directorio por módulo (Ventas, Cajas, Productos, Categorías, Gastos, Movimientos, Usuarios, Roles, Métodos de pago, Auditoría) con sus subcarpetas *Schemas* (formularios), *Tables* (listados) y *Pages* (listar, crear, editar).
- *app/Filament/Pages/* y *app/Filament/Widgets/*: página de reportes, página de consentimiento de datos, indicadores del escritorio y tabla de stock crítico.
- *app/Models/*, *app/Observers/* y *app/Policies/*: modelos del dominio, observadores que recalculan la caja y políticas de acceso por permiso.
- *database/migrations/* y *database/seeders/*: estructura de la base de datos y datos iniciales (roles y permisos, administrador, categorías, métodos de pago y los 42 productos de MundoCoco).
- *tests/Unit/* y *tests/Feature/*: 28 archivos con 160 pruebas automatizadas.

- *config/mundococo.php*: parámetros configurables sin tocar el código (rango de precios, factor de reorden, contacto para habeas data).

Una decisión de diseño transversal es el manejo del dinero: todos los cálculos se hacen en centavos enteros mediante el *trait* Dinero, y los valores se persisten como decimales con dos cifras. Así se evitan los errores de redondeo propios de los números de punto flotante en los totales de ventas y en el cuadro de caja.

Figura 17. *Conversión exacta de valores monetarios a centavos*

```
private function aCentavos(mixed $valor, string $campo = 'total'): int
{
    $normalizado = str_replace(',', '.', trim((string) $valor));
    if (! preg_match('/^([0-9]+)(?:\.[0-9]{1,2})?$/i', $normalizado, $coincidencias)) {
        throw ValidationException::withMessages([$campo => 'El valor monetario no es válido.']);
    }

    return ((int) $coincidencias[1] * 100) + (int) str_pad($coincidencias[2] ?? '', 2, '0');
}

private function desdeCentavos(int $centavos): string
{
    // Signo aparte: intdiv(-50, 100) es 0 y perdería el "-" de -0.50.
    $signo = $centavos < 0 ? '-' : '';
    $absoluto = abs($centavos);

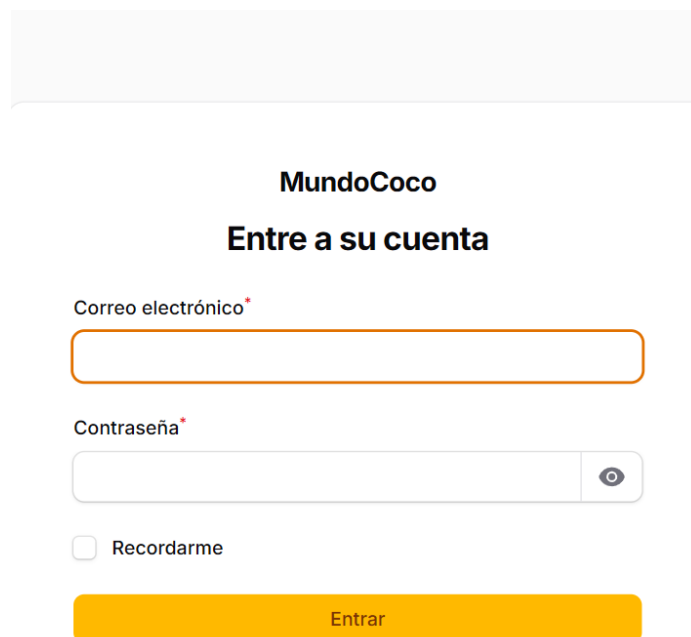
    return sprintf('%s%d.%02d', $signo, intdiv($absoluto, 100), $absoluto % 100);
}
```

Nota. Elaboración propia. Fragmento de *app/Support/Dinero.php*, líneas 18 a 35.

4.4.3 Inicio de Sesión, Roles y Permisos

La autenticación la provee el panel de Filament sobre el sistema de autenticación de Laravel: las contraseñas se almacenan cifradas con *bcrypt* y la sesión expira tras 120 minutos de inactividad. Si las credenciales no coinciden, el sistema muestra un mensaje de error sin revelar cuál de los dos datos es incorrecto.

Figura 18. *Interfaz de inicio de sesión*



The image shows a login interface for 'MundoCoco'. At the top, there is a light gray header bar. Below it, the text 'MundoCoco' is centered in a bold, black font. Underneath, the text 'Entre a su cuenta' is also centered in a bold, black font. The form consists of two input fields: 'Correo electrónico*' and 'Contraseña*'. The 'Correo electrónico*' field is a simple white box with an orange border. The 'Contraseña*' field is a white box with a gray border and a toggle icon (an eye) on the right side. Below the password field, there is a checkbox labeled 'Recordarme'. At the bottom of the form, there is a large orange button with the text 'Entrar' in white.

Nota. Elaboración propia.

Los permisos se definen en un único lugar, el *RoleSeeder*, que crea 31 permisos granulares y los asigna a los tres roles del documento de requisitos. Las políticas de cada módulo consultan permisos y no nombres de rol, de modo que el administrador puede crear roles nuevos desde la interfaz sin modificar el código.

Figura 19. *Asignación de permisos a los roles Administrador, Operario y Consultor*

```
$admin = Role::firstOrCreate(['name' => self::ADMIN, 'guard_name' => 'web']);
$operario = Role::firstOrCreate(['name' => self::OPERARIO, 'guard_name' => 'web']);
$consultor = Role::firstOrCreate(['name' => self::CONSULTOR, 'guard_name' => 'web']);

// Administrador: acceso total al sistema.
$admin->syncPermissions(Permission::all());

// Operario: registro de ventas y consulta de inventario.
$operario->syncPermissions([
    'ver ventas', 'crear ventas', 'editar ventas',
    'ver productos', 'ver categorias', 'ver movimientos',
]);

// Consultor: solo visualización de reportes.
$consultor->syncPermissions([
    'ver reportes', 'exportar reportes',
]);
```

Nota. Elaboración propia. Fragmento de database/seeder/RoleSeeder.php, líneas 58 a 74.

Figura 20. *Interfaz de gestión de usuarios*

The screenshot shows the 'Usuarios' management page in the MundoCoco application. The sidebar on the left contains navigation links for various system components. The main area features a table of users with the following data:

Nombre	Correo electrónico	Roles	Ver	Editar
Administrador MundoCoco	admin@mundococo.test	Admin	Ver	Editar
Operario Turno Mañana	operario@mundococo.test	Operario	Ver	Editar
Consultor Propietario	consultor@mundococo.test	Consultor	Ver	Editar

At the bottom of the table, it indicates 'Se muestran de 1 a 3 de 3 resultados' and a pagination control set to 'por página 10'.

Nota. Elaboración propia.

4.4.4 Productos y Categorías

El módulo de productos permite crear, consultar, editar, desactivar y eliminar productos.

La eliminación es lógica: el producto sale del catálogo, pero sus ventas y su historial de

movimientos se conservan y puede restaurarse. El listado muestra el stock en rojo cuando está en el mínimo o por debajo, y la columna *Stock calculado* aplica la fórmula del RF05 sobre el historial para detectar inconsistencias.

Figura 21. *Interfaz de listado de productos*

Código	Nombre	Categoría	Precio de venta	Precio de costo	Stock	Stock calculado	Mini
	Helado Arequipe	Helados	4.500,00 COP	2.000,00 COP	48	18	5
	Helado Bocadillo	Helados	4.500,00 COP	2.000,00 COP	37	12	5
	Helado Brownie	Helados	5.000,00 COP	2.200,00 COP	11	-7	5
	Helado Café	Helados	4.500,00 COP	2.000,00 COP	5	-7	5
	Helado Chicle	Helados	4.500,00 COP	2.000,00 COP	16	-4	8
	Helado Chocolate	Helados	4.500,00 COP	2.000,00 COP	2	-1	5
	Helado Fresa	Helados	4.500,00 COP	2.000,00 COP	19	-3	5
	Helado Guanábana	Helados	4.800,00 COP	2.100,00 COP	5	-10	5

Nota. Elaboración propia.

El formulario de producto valida el precio de venta dentro del rango definido en la configuración (RF01) y genera el código automáticamente cuando se deja vacío.

Figura 22. *Validación del precio de venta en el formulario de producto*

```

TextInput::make('precio_venta')
    ->label('Precio de Venta')
    ->numeric()
    ->prefix('$')
    ->minValue(fn (): int => config('mundococo.precio_venta_min'))
    ->maxValue(fn (): int => config('mundococo.precio_venta_max'))
    ->helperText(fn (): string => 'Entre $'.number_format(config('mundococo.precio_venta_min'), 0, ',',
    '.').' y $'.number_format(config('mundococo.precio_venta_max'), 0, ',', '.').'.')
    ->required(),

```

Nota. Elaboración propia. Fragmento de

app/Filament/Resources/Productos/Schemas/ProductoForm.php, líneas 36 a 43.

Figura 23. Interfaz de registro de producto

MundoCoco Q Buscar AM

Productos > Crear

Crear Producto

Categoría*

Código (RF01)

 Único en el catálogo. Se genera automáticamente si se deja vacío.

Nombre*

Descripción

Precio de Venta*

 RF01: entre \$2.000 y \$95.000.

Precio de Costo

Stock Actual*

Stock Mínimo (Alerta)*

☒ Activo

Nota. Elaboración propia.

Figura 24. Interfaz de categorías

MundoCoco Q Buscar AM

Categorías > Listado

Categorías

☐	Nombre	Descripción	Productos	Activa
☐	Helados	Helados artesanales: Arequipe, Bocadillo, Brownie,...	14	☒ Editar
☐	Bebidas	Limonada, Mandarindada, Cerezada, Avena, Cocofee,...	13	☒ Editar
☐	Aceites	Aceites de coco presentaciones 30ml, 60ml, 100ml,...	8	☒ Editar
☐	Productos de Coco	Leche de Coco, PET 500/1000, Galón, Tortas, Cocada...	7	☒ Editar

Se muestran de 1 a 4 de 4 resultados por página 10

Nota. Elaboración propia.

4.4.5 Movimientos de Inventario y Trazabilidad

Todo cambio de stock genera un registro en *movimientos_inventario* con su tipo, fecha de la transacción, cantidad, stock anterior y nuevo, usuario y motivo (RF12). Las entradas y salidas manuales se registran por lote: una compra de varios productos se guarda en una sola transacción y, si una línea es inválida, no se guarda ninguna. El stock calculado se obtiene con una única consulta SQL que suma entradas y resta salidas desde el último inventario inicial de cada producto.

Figura 25. *Cálculo del stock total a partir del historial de movimientos (RF05)*

```
public function stockCalculado(?array $productoIds = null): array
{
    $tabla = (new MovimientoInventario)->getTable();
    $ultimoInicial = MovimientoInventario::query()
        ->where('tipo', MovimientoInventario::TIPO_INICIAL)
        ->groupBy('producto_id')
        ->selectRaw('producto_id, MAX(id) as ultimo_id');

    $entradas = "'".implode("'", MovimientoInventario::TIPOS_ENTRADA)."'";

    return MovimientoInventario::query()
        ->leftJoinSub($ultimoInicial, 'base', 'base.producto_id', '=', "{$tabla}.producto_id")
        ->where(fn ($q) => $q->whereNull('base.ultimo_id')->orWhereColumn("{$tabla}.id", '>=',
'base.ultimo_id'))
        ->when($productoIds !== null, fn ($q) => $q->whereIn("{$tabla}.producto_id", $productoIds))
        ->groupBy("{$tabla}.producto_id")
        ->selectRaw("{$tabla}.producto_id as producto_id")
        ->selectRaw("SUM(CASE WHEN {$tabla}.tipo IN ({$entradas}) THEN {$tabla}.cantidad ELSE -
{$tabla}.cantidad END) as calculado")
        ->toBase()
        ->get()
        ->mapWithKeys(fn ($fila) => [(int) $fila->producto_id => (int) $fila->calculado])
        ->all();
}
```

Nota. Elaboración propia. Fragmento de `app/Services/InventarioService.php`, líneas 131 a 152.

Figura 26. Interfaz del historial de movimientos de inventario

MundoCoco Q Buscar AM

Operación diaria ^

- Ventas
- Cajas
- Gastos

Catálogo ^

- Productos
- Categorías
- Métodos de Pago
- Movimientos**

Informes ^

- Reportes
- Auditoría

Seguridad ^

- Usuarios

Movimientos De Inventario > Listado

Movimientos De Inventario Nuevo movimiento

Q Buscar 0

Id	Producto	Tipo	Cantidad	Anterior	Nuevo	Usuario	Motivo
39	Cerezada	Devolución	1	17	18	Administrador MundoCoco	Anulación venta #22: Clie
35	Helado Mani	Venta	2	12	10	Administrador MundoCoco	Venta #19
38	Cerezada	Venta	1	18	17	Administrador MundoCoco	Venta #22
32	Helado Chicle	Venta	1	17	16	Administrador MundoCoco	Venta #16
34	Helado Mangoviche	Venta	1	6	5	Administrador MundoCoco	Venta #18
37	Limonada	Venta	2	24	22	Administrador MundoCoco	Venta #21
31	Helado Brownie	Venta	2	15	13	Administrador MundoCoco	Venta #15
33	Helado Fresa	Venta	2	21	19	Administrador MundoCoco	Venta #17

Nota. Elaboración propia.

Figura 27. Interfaz de registro de movimiento de inventario por lote

MundoCoco Q Buscar AM

Operación diaria ^

- Ventas
- Cajas
- Gastos

Catálogo ^

- Productos
- Categorías
- Métodos de Pago
- Movimientos**

Informes ^

- Reportes
- Auditoría

Seguridad ^

- Usuarios

Movimientos De Inventario > Crear

Crear Movimiento

Movimiento de inventario

Tipo de Movimiento* Seleccione una opción

Fecha y hora de la transacción* sep 24, 2026 17:39

Motivo Compra proveedor X, Ajust

Observaciones

Productos

Lineas

Producto* Seleccione una opción

Cantidad* Debe ser positiva y mayor a cero

+ Agregar producto

Crear Crear y crear otro Cancelar

Nota. Elaboración propia.

4.4.6 Ventas

Las ventas se registran exclusivamente a través de *VentaService*: las páginas de creación y edición del panel solo delegan en él. El servicio valida que la fecha no sea futura y que la caja de esa fecha no esté cerrada, bloquea los productos involucrados en orden de identificador (*lockForUpdate*) para evitar que dos ventas simultáneas vendan el mismo stock, agrupa las líneas repetidas de un mismo producto antes de validar la existencia y recalcula precios, subtotales y total en el servidor, sin confiar en los valores enviados por el navegador. Todo ocurre dentro de una transacción con hasta tres reintentos.

Figura 28. Registro transaccional de una venta

```
public function crear(array $atributos, array $detalles): Venta
{
    return DB::transaction(function () use ($atributos, $detalles): Venta {
        $fechaVenta = Carbon::parse($atributos['fecha_venta'] ?? now());
        $this->asegurarFechaNoFutura($fechaVenta);
        $this->asegurarCajaNoCerrada($fechaVenta);
        $metodoPago = $this->obtenerMetodoPagoActivo($atributos['metodo_pago_id'] ?? null);
        [$productos, $cantidades] = $this->bloquearYValidarProductos($detalles);
        [$lineas, $totalCentavos] = $this->calcularLineas($detalles, $productos);

        $venta = Venta::query()->create([
            'metodo_pago_id' => $metodoPago->id,
            'total' => $this->desdeCentavos($totalCentavos),
            'observaciones' => $atributos['observaciones'] ?? null,
            'fecha_venta' => $fechaVenta,
        ]);

        $venta->detalles()->createMany($lineas);

        foreach ($cantidades as $productoId => $cantidad) {
            $this->moverStock($productos->get($productoId), -$cantidad, $venta, 'Venta #'.$venta->id,
            $fechaVenta);
        }

        AuditService::logVentaCreada($venta);
        $this->auditarPreciosModificados($venta, $lineas);

        return $venta->load(['detalles.producto', 'metodoPago']);
    }, 3);
}
```

Nota. Elaboración propia. Fragmento de `app/Services/VentaService.php`, líneas 33 a 61.

Figura 29. Bloqueo de productos y validación de stock agregado

```
// withTrashed: una edición puede conservar un producto eliminado después de venderlo.
$productos = Producto::withTrashed()
    ->whereIn('id', array_unique(array_merge(array_keys($cantidades), array_keys($reservadas))))
    ->orderBy('id')
    ->lockForUpdate()
    ->get()
    ->keyBy('id');

foreach ($cantidades as $productoId => $cantidad) {
    $producto = $productos->get($productoId);
    $reservada = $reservadas[$productoId] ?? 0;
    $vendible = $producto !== null && $producto->activo && ! $producto->trashed();

    if ($producto === null || (! $vendible && $reservada === 0)) {
        throw ValidationException::withMessages(['detalles' => 'Uno de los productos no existe o
está inactivo.']);
    }

    if ($cantidad > $producto->stock_actual + $reservada) {
        throw ValidationException::withMessages(['detalles' => "Stock insuficiente para {$producto-
>nombre}."]);
    }
}

return [$productos, $cantidades];
}
```

Nota. Elaboración propia. Fragmento de app/Services/VentaService.php, líneas 213 a 236.

Figura 30. Interfaz de listado de ventas con estado y método de pago

# Venta	Método de Pago	Total	Estado	Observaciones	Fecha
19	Efectivo	9.400,00 COP	Vigente		24/09/2026 11:55
22	Transferencia	4.500,00 COP	Anulada		24/09/2026 11:28
16	Tarjeta	4.500,00 COP	Vigente		24/09/2026 11:22
18	Nequi	4.800,00 COP	Vigente		24/09/2026 10:44
21	Efectivo	8.000,00 COP	Vigente		24/09/2026 10:17
15	Transferencia	10.000,00 COP	Vigente		24/09/2026 10:11
17	Efectivo	9.000,00 COP	Vigente		24/09/2026 09:33
20	Nequi	5.000,00 COP	Vigente		24/09/2026 09:06

Nota. Elaboración propia.

Figura 31. *Interfaz de registro de venta*

MundoCoco Buscar AM

Escritorio

Ventas > Crear

Crear Venta

Información de la Venta

Método de Pago* Seleccione una opción Fecha y hora de venta* sep 24, 2026 17:39

Total Venta \$ 0

Observaciones

Productos de la Venta

Detalles

Producto* Seleccione una opción

Cantidad*	Precio Unitario*	Subtotal
1	\$	\$

Se sugiere el precio base; puede ajustarse.

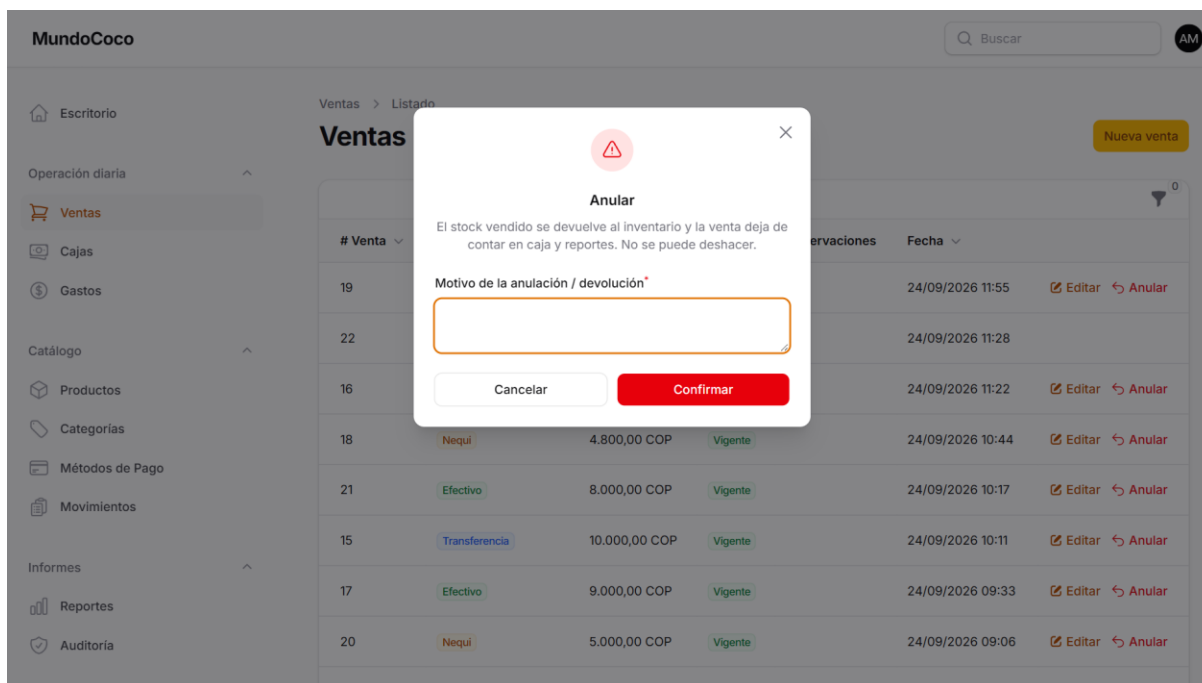
+ Agregar producto

Crear Crear y crear otro Cancelar

Nota. Elaboración propia.

Cuando una venta debe revertirse (devolución), el administrador la anula indicando el motivo: el stock se repone con movimientos de tipo devolución, la venta deja de sumar en la caja y en los reportes y queda marcada como anulada, sin eliminarse.

Figura 32. *Diálogo de anulación de venta*



Nota. Elaboración propia.

4.4.7 Caja y Gastos

La caja se calcula por la fecha de la venta (*fecha_venta*) y no por la fecha de registro, de modo que una venta registrada tarde queda en el día correcto. Los observadores de ventas y gastos recalculan automáticamente la caja abierta de la fecha afectada. Al cerrar, el administrador ingresa el dinero contado y el sistema calcula el saldo teórico ($\text{base} + \text{ventas} - \text{gastos}$) y la diferencia; desde ese momento la fecha queda bloqueada para ventas y gastos.

Figura 33. *Cierre de caja con cálculo del saldo teórico y la diferencia (RF11)*

```

public function cerrar(Caja $caja, array $datos): Caja
{
    return DB::transaction(function () use ($caja, $datos): Caja {
        $caja = Caja::query()->lockForUpdate()->findOrFail($caja->id);

        if ($caja->estado !== 'abierta') {
            throw ValidationException::withMessages(['caja' => 'La caja ya fue cerrada.']);
        }

        $totales = $this->totalesPorFecha($caja->fecha);
        $saldoReal = $this->aCentavos($datos['saldo_real'] ?? null, 'saldo_real');
        // RF11: Saldo Teórico = Base + Ventas - Gastos
        $esperado = $this->aCentavos($caja->saldo_inicial) + $this->aCentavos($totales['total']) -
        $this->aCentavos($totales['gastos']);

        $caja->fill([
            ...$this->camposTotales($totales),
            'saldo_teorico' => $this->desdeCentavos($esperado),
            'saldo_real' => $this->desdeCentavos($saldoReal),
            'diferencia' => $this->desdeCentavos($saldoReal - $esperado),
            'estado' => 'cerrada',
            'fecha_cierre' => now(),
            'observaciones_cierre' => $datos['observaciones_cierre'] ?? null,
        ])->saveQuietly();

        AuditService::logCajaCerrada($caja);

        return $caja;
    }, 3);
}

```

Nota. Elaboración propia. Fragmento de app/Services/CajaService.php, líneas 47 a 75.

Figura 34. *Interfaz de cajas con totales por método de pago*

MundoCoco Buscar AM

Escritorio Cajas > Listado Nueva caja

Cajas

Fecha	Estado	Saldo Inicial	Efectivo	Nequi	Transferencias	Tarjetas	Total Ventas
24/09/2026	abierta	120.000,00 COP	30.900,00 COP	9.800,00 COP	10.000,00 COP	4.500,00 COP	55.200,00 COP
23/09/2026	cerrada	100.000,00 COP	109.600,00 COP	68.600,00 COP	26.900,00 COP	41.000,00 COP	246.100,00 COP

Se muestran de 1 a 2 de 2 resultados por página 10

Nota. Elaboración propia.

Figura 35. *Diálogo de cierre de caja*

MundoCoco Buscar AM

Escritorio Cajas > Listado Nueva caja

Cajas

Cerrar caja
¿Está segura/o de hacer esto?

Dinero contado*

\$

Observaciones del cierre

Fecha	Estado	Saldo Inicial	Efectivo	Nequi	Transferencias	Tarjetas	Total Ventas
24/09/2026	abierta	120.000,00 COP	30.900,00 COP	9.800,00 COP	10.000,00 COP	4.500,00 COP	55.200,00 COP
23/09/2026	cerrada	100.000,00 COP	109.600,00 COP	68.600,00 COP	26.900,00 COP	41.000,00 COP	246.100,00 COP

Se muestran de 1 a 2 de 2 resultados por página 10

Nota. Elaboración propia.

Figura 36. Interfaz de gastos

MundoCoco Q Buscar AM

Escritorio Gastos > Listado

Gastos Nuevo gasto

Operación diaria ^

Ventas

Cajas

Gastos

Catálogo ^

Productos

Categorías

Métodos de Pago

Movimientos

Informes ^

Reportes

Auditoría

Q Buscar 0 Filter Menu

Fecha	Descripción	Categoría	Monto	Registrado por
24/09/2026	Pago servicio de energía	servicios	30.000,00 COP	Administrador MundoCoco Editar
23/09/2026	Domicilio proveedor	transporte	8.000,00 COP	Administrador MundoCoco
23/09/2026	Compra de coco rallado	materia_prima	45.000,00 COP	Administrador MundoCoco

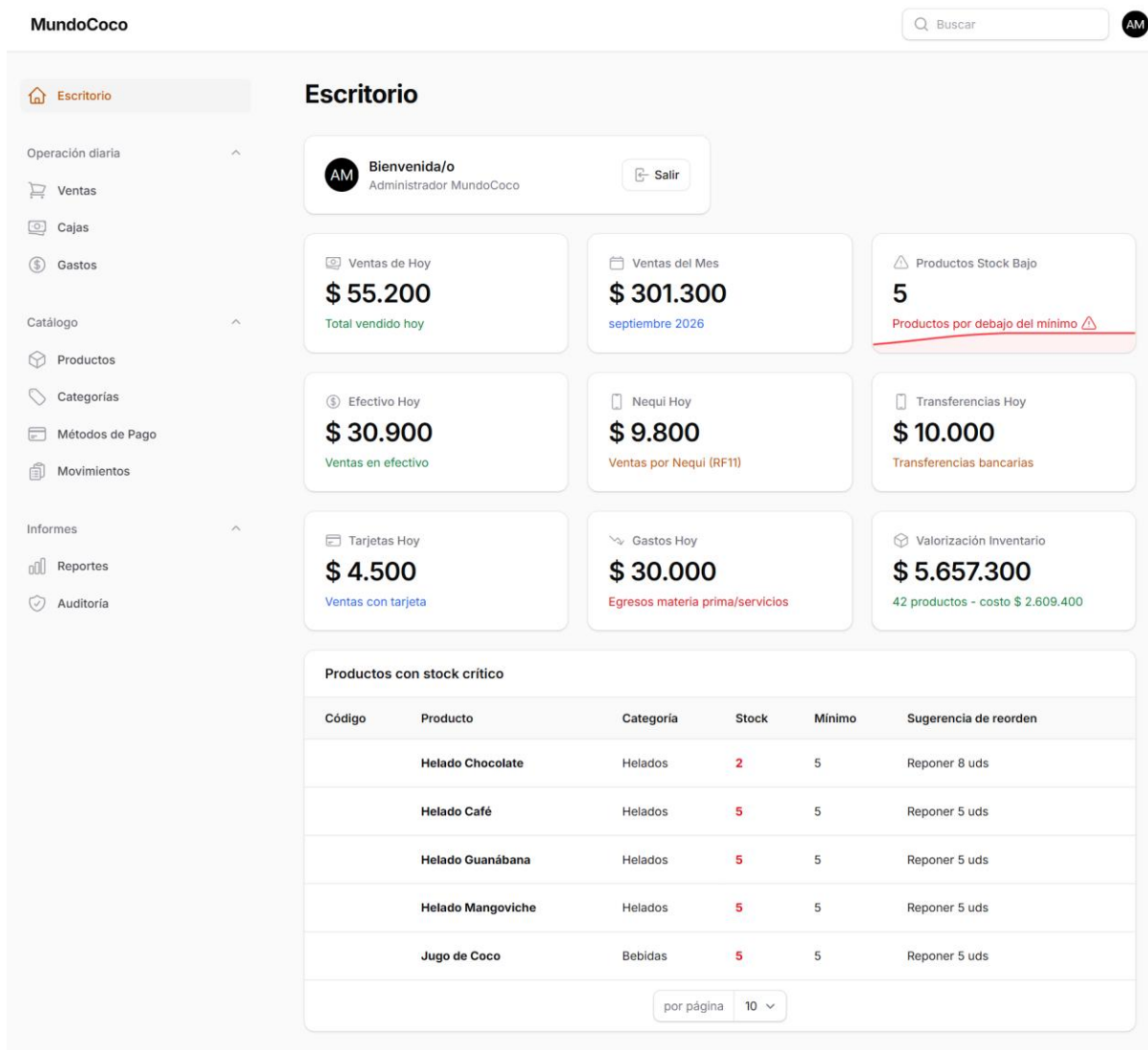
Se muestran de 1 a 3 de 3 resultados por página 10

Nota. Elaboración propia.

4.4.8 Escritorio, Alertas y Reportes

El escritorio presenta los indicadores del día (ventas del día y del mes, ventas por método de pago, gastos y valorización del inventario) y la tabla de productos con stock crítico, con la sugerencia de reorden calculada como el stock mínimo multiplicado por un factor configurable, menos el stock actual (RF07). La página de reportes permite elegir un periodo y consultar once reportes: valorización, indicadores de gestión, inventario por categoría, stock bajo, comparativo de periodos, movimientos, ventas diarias por producto, ventas por categoría, productos más vendidos, ingresos por día y flujo de efectivo. Cada uno se puede descargar en PDF, Excel o CSV.

Figura 37. *Escritorio con indicadores y productos con stock crítico*



Nota. Elaboración propia.

Figura 38. Interfaz de reportes

Reportes MundoCoco

Inventario, ventas y caja del 2026-08-25 al 2026-09-24. Ajusta el periodo y pulsa «Aplicar periodo».

Periodo del reporte

Filtra ventas, movimientos e indicadores por fecha. El inventario muestra el estado actual; el flujo de caja es del día «Hasta».

Desde

Hasta

Aplicar periodo

Valorización de inventario

Cantidad x precio de los 42 productos activos.

PDF

Excel

CSV

A costo

A precio de venta

Ganancia potencial

Productos en stock bajo

\$ 2.609.400

\$ 5.657.300

\$ 3.047.900

5

Indicadores de gestión

Nota. Elaboración propia.

Figura 39. Reporte de valorización de inventario exportado a PDF

MundoCoco - Valorización de inventario	
Periodo: 2026-08-25 al 2026-09-24 Generado: 24/09/2026 17:40	
Concepto	Valor
Productos activos	42
Valorización a costo (cantidad x costo)	2609400.00
Valorización a venta (cantidad x precio)	5657300.00
Ganancia potencial	3047900.00

Nota. Elaboración propia.

4.4.9 Seguridad, Auditoría y Respaldos

El servicio de auditoría registra las operaciones críticas —creación, edición y anulación de ventas, precios distintos al base, todo cambio de stock, cierres de caja y aceptación de la política de datos— con el usuario, la dirección IP, el navegador, los cambios y los valores anteriores. La

pantalla de auditoría es de solo lectura y únicamente la ve el administrador. Adicionalmente, una tarea programada genera cada día a las 2:00 a. m. un volcado SQL de la base de datos, con retención de 30 días, y otra elimina semanalmente los registros de auditoría con más de un año.

Figura 40. Registro de una operación en la auditoría

```
public static function log(string $accion, ?Model $modelo = null, ?array $cambios = null, ?array
$anteriores = null): AuditLog
{
    return AuditLog::create([
        'user_id' => Auth::id(),
        'accion' => $accion,
        'modelo_type' => $modelo ? get_class($modelo) : null,
        'modelo_id' => $modelo?->getKey(),
        'cambios' => $cambios,
        'valores_anteriores' => $anteriores,
        'ip_address' => Request::ip(),
        'user_agent' => Request::userAgent(),
    ]);
}
```

Nota. Elaboración propia. Fragmento de app/Services/AuditService.php, líneas 12 a 24.

Figura 41. Tareas programadas de respaldo diario y limpieza de auditoría

```
// RNF04/RNF05 Seguridad y confiabilidad: respaldo diario 2am + limpieza logs.
// El comando backup:database genera un volcado SQL real en storage/app/backups
// (ver App\Console\Commands\BackupDatabase) y purga copias de más de 30 días.
Schedule::command('backup:database')->dailyAt('02:00')->onFailure(function () {
    logger()->warning('Backup diario falló');
});

// Limpieza auditoría > 365 días
Schedule::call(function () {
    AuditLog::where('created_at', '<', now()->subDays(365))->delete();
})->weekly();
```

Nota. Elaboración propia. Fragmento de routes/console.php, líneas 12 a 22.

Figura 42. Interfaz de auditoría

The screenshot shows the 'Auditoría' (Audit) interface of the MundoCoco system. The left sidebar contains navigation links for Ventas, Cajas, Gastos, Catálogo, Productos, Categorías, Métodos de Pago, Movimientos, Informes, Reportes, Auditoría (highlighted), Seguridad, Usuarios, and Roles. The main content area displays a table of audit records with columns: Fecha, Usuario, Acción, Registro, and Detalle. The table contains 8 rows of data, all performed by 'Administrador MundoCoco' on '24/09/2026 17:36:41'. The actions include 'stock_ajuste' (3 times) and 'venta_anulada' and 'venta_creada' (5 times). The details column contains JSON objects representing the audit data.

Fecha	Usuario	Acción	Registro	Detalle
24/09/2026 17:36:41	Administrador MundoCoco	stock_ajuste	Producto #22	{"stock":5,"tipo":"ajuste_negativo"}
24/09/2026 17:36:41	Administrador MundoCoco	stock_ajuste	Producto #8	{"stock":5,"tipo":"ajuste_negativo"}
24/09/2026 17:36:41	Administrador MundoCoco	stock_ajuste	Producto #4	{"stock":5,"tipo":"ajuste_negativo"}
24/09/2026 17:36:41	Administrador MundoCoco	stock_ajuste	Producto #3	{"stock":11,"tipo":"merma"}
24/09/2026 17:36:41	Administrador MundoCoco	venta_anulada	Venta #22	{"motivo":"Cliente devolvió el producto","total":"4500.00"}
24/09/2026 17:36:41	Administrador MundoCoco	venta_creada	Venta #22	{"metodo_pago_id":3,"total":"4500.00","observaciones":null}
24/09/2026 17:36:41	Administrador MundoCoco	venta_creada	Venta #21	{"metodo_pago_id":1,"total":"8000.00","observaciones":null}
24/09/2026 17:36:40	Administrador MundoCoco	venta_creada	Venta #20	{"metodo_pago_id":2,"total":"5000.00","observaciones":null}

Nota. Elaboración propia.

4.4.10 Protección de Datos Personales

Para cumplir la Ley 1581 de 2012, el sistema publica la política de tratamiento de datos en la ruta */privacidad*, enlazada desde la pantalla de ingreso. Un *middleware* impide operar a cualquier usuario que no haya aceptado la política: lo redirige a la página de consentimiento y, al aceptar, registra la fecha como prueba de la autorización.

Figura 43. *Middleware que exige el consentimiento para el tratamiento de datos*

```
public function handle(Request $request, Closure $next): Response
{
    $usuario = $request->user();
    $urlConsentimiento = ConsentimientoDatos::getUrl();

    $permitida = $request->url() === $urlConsentimiento || $request->routeIs('filament.*.auth.logout');

    if ($usuario && ! $usuario->aceptoPolitica() && ! $permitida) {
        return redirect()->to($urlConsentimiento);
    }

    return $next($request);
}
```

Nota. Elaboración propia. Fragmento de `app/Http/Middleware/ExigirConsentimientoDatos.php`, líneas 17 a 29.

4.4.11 Escalabilidad e Integración

Para atender el RNF07, las ventas, cajas, gastos y movimientos se asocian a una sucursal (por defecto, la principal), lo que deja preparado el sistema para una futura expansión a varios puntos de venta. Además, se expone una API REST de solo lectura protegida con tokens de Laravel Sanctum, que permite consultar productos, ventas y reportes desde sistemas externos (por ejemplo, contables) respetando los permisos del usuario dueño del token, con un límite de 60 solicitudes por minuto.

4.5 Fase de Pruebas

El plan de pruebas se orientó por la norma ISO/IEC/IEEE 29119 y combinó cuatro niveles: pruebas automatizadas unitarias y de integración, pruebas funcionales sobre los requisitos, pruebas de rendimiento y pruebas heurísticas de usabilidad. Las pruebas automatizadas se ejecutan en una base de datos SQLite en memoria que se reinicia en cada prueba, de modo que son independientes entre sí y no requieren datos reales.

4.5.1 Pruebas Unitarias y de Integración Automatizadas

La suite automatizada cuenta con 160 pruebas y 566 aserciones distribuidas en 28 archivos: 62 pruebas unitarias sobre servicios, modelos, observadores y utilidades, y 98 pruebas de funcionalidad que recorren flujos completos, incluido el uso del panel de Filament. En la última ejecución, todas las pruebas finalizaron satisfactoriamente en 12,47 segundos. Las Tablas siguientes resumen su alcance.

Tabla 23. *Pruebas unitarias automatizadas*

Archivo	Pruebas	Qué verifica
Unit/Support/DineroTest	3	Conversión exacta a centavos y conservación del signo en montos menores a \$1.
Unit/Models (7 archivos)	35	Relaciones, conversiones y reglas de los modelos Caja, Categoría, Método de pago, Producto (incluido el código único), Venta y Detalle de venta.
Unit/Observers (3 archivos)	7	Recalculo de la caja al crear o editar ventas y gastos; trazado de ajustes manuales de stock.
Unit/Services/VentaServiceTest	4	Precio confiable del servidor, rechazo de stock acumulado insuficiente con líneas duplicadas y de ventas en caja cerrada.
Unit/Services/VentaAlertaStockTest	4	Alerta de stock mínimo después de una venta (RF04).
Unit/Services/CajaServiceTest	2	Una sola caja abierta por fecha; cierre con diferencia calculada por fecha de venta.
Unit/Services/InventarioServiceTest	7	Inventario inicial, entradas, salidas, validaciones, cálculo RF05 y valorización.

Nota. Elaboración propia.

Tabla 24. *Pruebas de funcionalidad e integración automatizadas*

Archivo	Pruebas	Qué verifica
CumplimientoAnteproyectoTest	35	Verificación punto por punto de los RF y RNF: métodos de pago con Nequi, precio aplicado auditado, fecha y hora de venta, roles estrictos, anulación, borrado lógico, lotes de inventario, flujo de caja, indicadores, consentimiento Ley 1581, sucursales y API.
InventarioFlowTest	9	Flujos completos de gestión de inventario.
ReporteExportTest	8	Exportación de todos los reportes en PDF, Excel y CSV, autenticación y validación de fechas.
ReportesServiceTest	8	Reportes de inventario y ventas (RF08, RF09).
CajasFlowTest	7	Apertura, totales por método de pago, cierre y diferencia.

Archivo	Pruebas	Qué verifica
VentasPanelTest	6	Ventas desde el panel Filament: precio del servidor, sobreventa sin registros a medias, caja cerrada, edición por diferencia neta.
CajaConGastosTest	5	Saldo teórico con gastos, faltantes y sobrantes, bloqueo de gastos en caja cerrada (RF11).
VentasFlowTest	5	Flujo de venta completo.
TrazabilidadTest	4	Movimientos con stock anterior y nuevo, filtros del historial, stock nunca negativo (RF12).
AdminUserSeederTest	4	Creación del administrador inicial desde la configuración.
SimulacionRealMundoCocoTest	3	Jornada completa de la tienda con categorías reales, 15 ventas en tres métodos de pago, gasto y cierre con diferencia cero.
BackupDatabaseTest	2	Volcado SQL real y purga por retención (RNF04).
RendimientoTest	2	Operaciones con 200 productos en menos de 3 segundos (RNF01).

Nota. Elaboración propia.

Las pruebas protegen las reglas críticas del negocio: una venta nunca deja stock negativo, ni siquiera con líneas repetidas del mismo producto; el cliente no puede definir el precio, el subtotal ni el total; las ventas de una fecha con caja cerrada se rechazan; la caja usa exclusivamente la fecha de la venta; y la diferencia solo se calcula con el dinero contado al cierre.

La integración continua ejecuta, además, la medición de cobertura sobre el código crítico (servicios, utilidades, observadores, políticas, *middleware* y modelos) y falla si es inferior al 80 %, en cumplimiento del RNF10. [COMPLETAR: porcentaje de cobertura reportado por la última ejecución de GitHub Actions].

4.5.2 Pruebas Funcionales

Las pruebas funcionales verifican el comportamiento del sistema frente a cada requisito, desde la perspectiva del usuario. La Tabla siguiente presenta los casos ejecutados; cada uno está respaldado por una o más pruebas automatizadas (ver Anexo 5) y fue verificado manualmente en el panel.

Tabla 25. Casos de prueba funcionales

ID	Caso de prueba	Procedimiento	Resultado esperado	Resultado
PF-01	Ingreso con credenciales válidas	Ingresar correo y contraseña de un usuario con rol.	Se muestra el escritorio con los módulos de su rol.	Aprobada
PF-02	Ingreso con credenciales inválidas	Ingresar una contraseña incorrecta.	Mensaje “Estas credenciales no coinciden con nuestros registros”; no se inicia sesión.	Aprobada
PF-03	Consentimiento de datos	Ingresar con un usuario que no ha aceptado la política.	Redirección a la página de consentimiento; tras aceptar, se guarda la fecha y se permite operar.	Aprobada
PF-04	Restricción por rol	Ingresar como Operario e intentar abrir Usuarios.	El menú no muestra el módulo y el acceso directo responde 403 Prohibido.	Aprobada
PF-05	Crear producto	Registrar un producto con stock 10 y sin código.	Se genera el código único y se registra el movimiento de inventario inicial.	Aprobada
PF-06	Precio fuera de rango	Registrar un producto con precio \$1.000.	El formulario rechaza el valor (rango \$2.000 – \$95.000).	Aprobada
PF-07	Eliminar y restaurar producto	Eliminar un producto con ventas y luego restaurarlo.	Sale del catálogo sin perder su historial y vuelve al restaurarlo.	Aprobada
PF-08	Compra de varios productos	Registrar una compra con tres líneas.	Se actualiza el stock de los tres productos y se crean tres movimientos.	Aprobada
PF-09	Lote con línea inválida	Registrar una merma que supera el stock en una de las líneas.	No se guarda ninguna línea; se muestra la validación.	Aprobada
PF-10	Venta con stock suficiente	Vender dos productos pagando con Nequi.	Se registra la venta, se descuenta el stock y la caja suma el valor en Nequi.	Aprobada
PF-11	Venta sin stock suficiente	Vender más unidades de las disponibles (incluso repartidas en dos líneas).	Mensaje “Stock insuficiente para [producto]”; no se registra nada.	Aprobada
PF-12	Venta con precio modificado	Cambiar el precio de una línea dentro del rango.	Se guarda el precio base y el aplicado, y se registra en la auditoría.	Aprobada
PF-13	Venta con fecha futura	Registrar una venta con fecha de mañana.	La venta se rechaza.	Aprobada
PF-14	Alerta de stock mínimo	Vender hasta dejar un producto en su mínimo.	Aparece la alerta y el producto se lista en el escritorio con su sugerencia de reorden.	Aprobada
PF-15	Editar venta	Reducir la cantidad de una venta de la caja abierta.	El stock se ajusta solo por la diferencia (devolución) y la caja se recalcula.	Aprobada
PF-16	Anular venta	Anular una venta indicando el motivo.	El stock se repone, la venta sale de la caja y de los reportes y queda auditada.	Aprobada
PF-17	Abrir caja duplicada	Abrir una segunda caja para la misma fecha.	Mensaje “Ya existe una caja abierta para esta fecha”.	Aprobada
PF-18	Gasto en la caja	Registrar un gasto de materia prima de \$30.000.	El saldo teórico de la caja disminuye en \$30.000.	Aprobada

ID	Caso de prueba	Procedimiento	Resultado esperado	Resultado
PF-19	Cierre de caja con faltante	Cerrar la caja contando \$1.000 menos del saldo teórico.	Diferencia de -\$1.000; la caja queda cerrada.	Aprobada
PF-20	Venta en caja cerrada	Registrar o editar una venta en una fecha con caja cerrada.	La operación se rechaza.	Aprobada
PF-21	Exportar reportes	Descargar cada reporte en PDF, Excel y CSV.	Se descargan archivos válidos con los datos del periodo.	Aprobada
PF-22	Exportación con fechas inválidas	Solicitar un reporte con “hasta” anterior a “desde”.	Respuesta 422 con el mensaje de rango inválido.	Aprobada
PF-23	Historial filtrado	Filtrar movimientos por producto, tipo y rango de fechas.	Solo se listan los movimientos que cumplen los filtros.	Aprobada
PF-24	Respaldo diario	Ejecutar el comando de respaldo.	Se genera el archivo SQL y se purgan los respaldos de más de 30 días.	Aprobada

Nota. Elaboración propia.

Figura 44. *Validación de credenciales inválidas en el inicio de sesión*

The screenshot shows the login interface for 'MundoCoco'. At the top, the text 'MundoCoco' is displayed above the heading 'Entre a su cuenta'. There are two input fields: 'Correo electrónico*' and 'Contraseña*'. The email field contains 'admin@mundococo.test' and is highlighted with a red border. Below the email field, a red error message states: 'Estas credenciales no coinciden con nuestros registros.' The password field is masked with dots and has a toggle icon on the right. Below the password field is a checkbox labeled 'Recordarme'. At the bottom is a yellow button labeled 'Entrar'.

Nota. Elaboración propia.

Figura 45. Validación de campos obligatorios en el registro de venta

MundoCoco Buscar AM

Crear Venta

Información de la Venta

Método de Pago* Fecha y hora de venta*

Seleccione una opción sep 24, 2026 17:39

Total Venta

\$ 0

Observaciones

Productos de la Venta

Detalles

Producto*

Seleccione una opción

Cantidad* Precio Unitario* Subtotal

1 \$ Subtotal

Completa este campo puede ajustarse.

+ Agregar producto

Crear **Crear y crear otro** **Cancelar**

Nota. Elaboración propia.

Figura 46. Escritorio del rol Operario: solo ventas y consulta de inventario

MundoCoco Buscar OM

Escritorio

Bienvenida/o
Operario Turno Mañana Salir

Ventas de Hoy
\$ 55.200
Total vendido hoy

Ventas del Mes
\$ 301.300
septiembre 2026

Productos Stock Bajo
5
Productos por debajo del mínimo

Efectivo Hoy
\$ 30.900
Ventas en efectivo

Nequi Hoy
\$ 9.800
Ventas por Nequi (RF11)

Transferencias Hoy
\$ 10.000
Transferencias bancarias

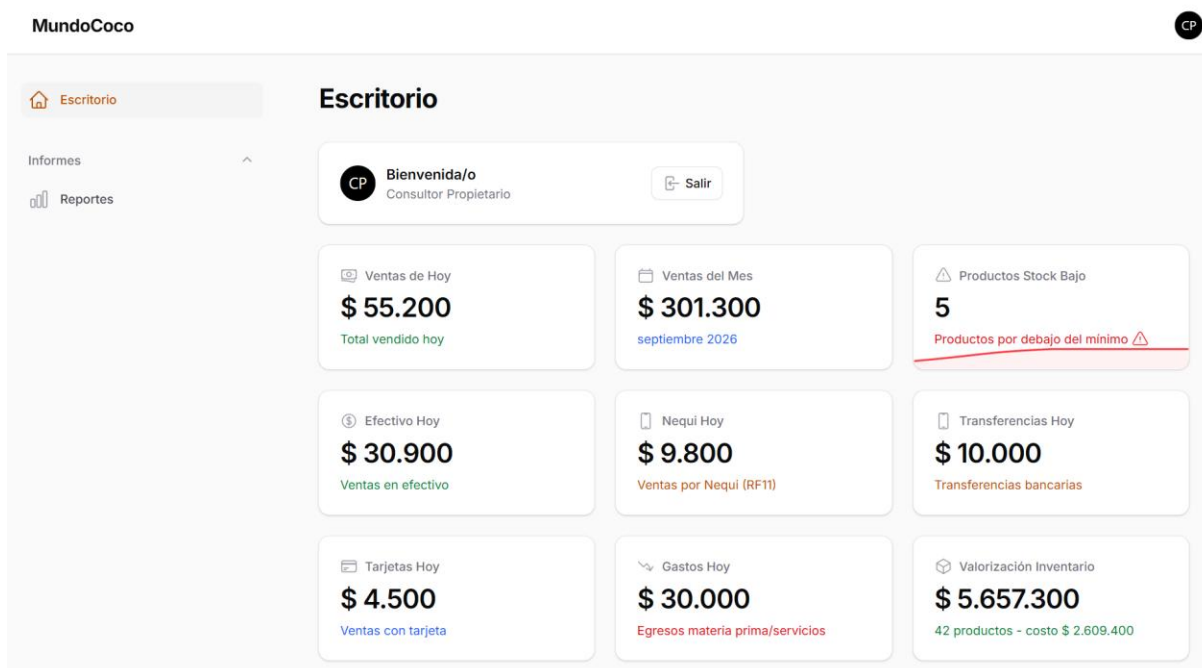
Tarjetas Hoy
\$ 4.500
Ventas con tarjeta

Gastos Hoy
\$ 30.000
Egresos materia prima/servicios

Valorización Inventario
\$ 5.657.300
42 productos - costo \$ 2.609.400

Nota. Elaboración propia.

Figura 47. *Escritorio del rol Consultor: solo indicadores y reportes*



Nota. Elaboración propia.

Figura 48. *Acceso denegado al Operario en el módulo de usuarios*

403 | Prohibido

Nota. Elaboración propia.

4.5.3 Pruebas de Rendimiento

El RNF01 exige un tiempo de respuesta máximo de 3 segundos con al menos 200 productos. Las pruebas automatizadas de rendimiento generan 200 productos y miden los reportes de inventario por categoría, stock bajo y valorización; la simulación de una jornada completa de la tienda (apertura de caja, compras, 15 ventas, gastos y cierre) también se mide contra ese límite.

Adicionalmente, se midió el tiempo de carga de las pantallas principales en el entorno local con datos de demostración (cinco cargas por pantalla).

Tabla 26. *Resultados de las pruebas de rendimiento*

Prueba	Límite	Resultado obtenido
Reportes de inventario con 200 productos (prueba completa, incluye la creación de datos)	< 3 s	0,26 s
Valorización del inventario con 200 productos	< 3 s	0,16 s
Simulación de jornada completa (15 ventas, gasto y cierre)	< 3 s	0,12 s
Carga del escritorio (mediana)	< 2 s (RNF09)	0,41 s
Carga del listado de productos (mediana)	< 2 s	0,45 s
Carga del listado de ventas (mediana)	< 2 s	0,33 s
Carga del formulario de nueva venta (mediana)	< 2 s	0,26 s
Carga de cajas / movimientos / reportes (mediana)	< 2 s	0,23 s / 0,30 s / 0,29 s

Nota. Elaboración propia. Tiempos medidos en el equipo de desarrollo con el servidor local de Laravel; en producción dependen del servidor y de la red.

4.5.4 Pruebas Heurísticas

La usabilidad de la interfaz se evaluó con las diez heurísticas de Nielsen (1994), revisando cada una sobre las pantallas del sistema.

Tabla 27. *Evaluación heurística de usabilidad*

Heurística	Evidencia en el sistema	Resultado
1. Visibilidad del estado del sistema	Notificaciones de éxito o error tras cada acción; estados de venta (Vigente/Anulada) y de caja (Abierta/Cerrada) con colores.	Cumple
2. Coincidencia entre el sistema y el mundo real	Terminología de la tienda: caja, base, cuadro, gastos, merma; menú agrupado en Operación diaria, Catálogo, Informes y Seguridad; interfaz en español.	Cumple
3. Control y libertad del usuario	Botones Cancelar en formularios y diálogos; ventas editables mientras la caja esté abierta; productos eliminados restaurables.	Cumple
4. Consistencia y estándares	Todos los módulos comparten la misma estructura de listado, formulario, filtros y acciones del panel.	Cumple
5. Prevención de errores	Diálogos de confirmación para cerrar caja y anular ventas; validación de stock, precios y fechas antes de guardar; la fecha cerrada queda bloqueada.	Cumple

Heurística	Evidencia en el sistema	Resultado
6. Reconocer antes que recordar	Selectores con búsqueda de productos por nombre o código; precio base sugerido; ayudas bajo los campos (rango de precios, stock mínimo).	Cumple
7. Flexibilidad y eficiencia de uso	Búsqueda global, filtros por columna, opción “Crear y crear otro” y registro de varios productos por operación.	Cumple
8. Diseño estético y minimalista	Interfaz limpia de Filament; el menú solo muestra los módulos permitidos al rol.	Cumple
9. Ayudar a reconocer, diagnosticar y recuperarse de errores	Mensajes específicos en español junto al campo con error (por ejemplo, “Stock insuficiente para Helado Chocolate”).	Cumple
10. Ayuda y documentación	Textos de ayuda en los formularios, manual de usuario y manual técnico del sistema.	Cumple

Nota. Elaboración propia.

4.5.5 Pruebas con el Usuario Final

Para validar el sistema en la operación real de la tienda (objetivo específico 4) se capacitó al personal y se aplicó la encuesta de satisfacción del Anexo 2. [COMPLETAR: fecha y duración del periodo de pruebas en la tienda, número de participantes y resultados de la encuesta (por ejemplo, porcentaje que calificó el sistema como fácil de usar, utilidad de las alertas y claridad de los reportes)].

Los indicadores de gestión que ofrece el sistema en la página de reportes —precisión del inventario (productos cuyo stock registrado coincide con el calculado), pérdidas valorizadas por mermas y ajustes, productos agotados, cajas cerradas con diferencia y ventas anuladas— permiten medir en operación real la precisión del inventario y la reducción de pérdidas. [COMPLETAR: valores de los indicadores medidos durante las pruebas con el cliente y comparación con la situación inicial].

4.6 Resultados Esperados

Al inicio del proyecto se plantearon los siguientes resultados esperados: el registro de productos con trazabilidad y alertas de stock en tiempo real; la integración del módulo de ventas

con el cuadro de caja, incluyendo la categorización de métodos de pago; la reducción de errores humanos y de tiempos operativos; la disponibilidad de información en tiempo real para tomar mejores decisiones; y la generación automatizada de reportes financieros y operativos. Como impactos se esperaban la disminución de las pérdidas económicas, la mejora de la eficiencia operativa, una mayor competitividad y posibilidad de escalar el negocio y el fortalecimiento de las capacidades técnicas del equipo en desarrollo ágil.

5. Resultados

A continuación se presentan los resultados obtenidos frente a cada objetivo específico.

Objetivo 1: levantamiento de requisitos. Mediante entrevistas con la administración de MundoCoco, el análisis de los formatos manuales y del archivo de Excel de control de caja y la observación del cierre de la jornada, se especificaron doce requisitos funcionales y diez no funcionales siguiendo las recomendaciones de la norma ISO/IEC/IEEE 29148. Estos requisitos se tradujeron en un Product Backlog de dieciséis historias de usuario priorizadas. Durante el análisis se resolvieron con el cliente las diferencias entre los documentos iniciales, en particular los métodos de pago y el manejo de precios distintos al precio base. [COMPLETAR: porcentaje del personal entrevistado y duración del levantamiento].

Objetivo 2: diseño de la arquitectura. Se diseñó una arquitectura por capas con tres módulos centrales —inventario, ventas y caja— y módulos de soporte para reportes, seguridad y auditoría. El diseño se documentó con el modelo entidad-relación, el diccionario de datos, diagramas de clases, diez casos de uso extendidos, cinco diagramas de secuencia y tres diagramas de actividades. La decisión de concentrar las reglas de negocio en servicios transaccionales permitió que las pantallas y la API compartan exactamente las mismas validaciones.

Objetivo 3: desarrollo de las funcionalidades. Se desarrolló el aplicativo web con Laravel 13 y Filament 5. El sistema registra las ventas con descuento automático del stock y trazabilidad de cada movimiento, calcula la caja diaria separando efectivo, Nequi, transferencias y tarjetas, descuenta los gastos del día y obtiene la diferencia del cuadre a partir del dinero contado. Se complementó con alertas de stock mínimo, once reportes exportables, auditoría de operaciones críticas, respaldo diario de la base de datos, consentimiento para el tratamiento de datos personales,

preparación para múltiples sucursales y una API de integración. La Tabla siguiente resume el cumplimiento de los requisitos.

Tabla 28. *Cumplimiento de los requisitos*

Requisito	Implementación	Estado
RF01 Productos	Módulo Productos con código único, rango de precio y borrado lógico	Cumplido
RF02 Inventario inicial	Movimiento tipo inicial con fecha de registro	Cumplido
RF03 Entradas	Movimientos por lote (compra, devolución, ajuste positivo)	Cumplido
RF04 Ventas	VentaService: stock, total en servidor, alerta de mínimo, precio aplicado auditado	Cumplido
RF05 Stock total	Columna Stock calculado e indicador de precisión	Cumplido
RF06 Categorías	Cuatro categorías de MundoCoco y 42 productos iniciales	Cumplido
RF07 Alertas	Widget de stock crítico con sugerencia de reorden	Cumplido
RF08 Reportes de inventario	Página de reportes con exportación PDF, Excel y CSV	Cumplido
RF09 Reportes de ventas	Ventas diarias, por categoría, más vendidos, ingresos y comparativo	Cumplido
RF10 Usuarios y roles	Administrador, Operario y Consultor con 31 permisos	Cumplido
RF11 Caja diaria	Caja por método de pago con gastos, cierre y flujo de efectivo	Cumplido
RF12 Trazabilidad	Historial con usuario, tipo, fecha y stock antes/después, con filtros	Cumplido
RNF01 – RNF10	Rendimiento, usabilidad, compatibilidad, seguridad, confiabilidad, mantenibilidad, escalabilidad, portabilidad, eficiencia y cumplimiento	Cumplido (ver secciones 4.4 y 4.5)

Nota. Elaboración propia.

Objetivo 4: validación del desempeño. Se ejecutó una suite de 160 pruebas automatizadas con resultado satisfactorio, veinticuatro casos de prueba funcionales, pruebas de rendimiento con tiempos muy inferiores a los límites exigidos (operaciones críticas por debajo de 0,3 segundos frente a un límite de 3 segundos y cargas de pantalla por debajo de 0,6 segundos) y una evaluación heurística de usabilidad. El sistema incorpora los indicadores de precisión de inventario, pérdidas y cuadre de caja definidos en el anteproyecto. **[COMPLETAR: resultados de la validación en operación real y de la encuesta de satisfacción].**

Como entregables complementarios se elaboraron el manual de usuario (Anexo 4), el manual técnico de instalación y mantenimiento y el presente documento. [COMPLETAR: forma de despliegue acordada con la tienda (servidor local u hosting), capacitación realizada y acuerdos de soporte].

6. Conclusiones y Observaciones

El levantamiento de información con la administración de MundoCoco permitió comprender que el problema central de la tienda no era solo el inventario, sino su relación con las ventas y con el cuadre de caja diario. Integrar los tres procesos en un mismo sistema, con la venta como punto que descuenta el stock y alimenta la caja, fue la decisión que dio sentido al proyecto. La norma ISO/IEC/IEEE 29148 resultó útil para redactar requisitos verificables; aun así, los documentos iniciales presentaban contradicciones (métodos de pago, manejo del precio) que solo pudieron resolverse conversando con el cliente, lo que confirma la importancia de validar los requisitos de forma continua y no solo al inicio.

En cuanto al diseño, concentrar la lógica de negocio en servicios transaccionales, y no en las pantallas, fue determinante para la confiabilidad del sistema. Reglas como impedir el stock negativo aun con líneas repetidas, recalcular los precios en el servidor o calcular la caja por la fecha de la venta se implementaron una sola vez y se aplican de la misma forma desde el panel, las pruebas y la API. El manejo del dinero en centavos enteros eliminó los errores de redondeo que habrían afectado precisamente el cuadre de caja, que era el principal síntoma del problema.

El desarrollo con Scrum permitió entregar incrementos funcionales y ajustar el alcance según la retroalimentación de la tienda. Funcionalidades que no estaban explícitas al inicio — como la anulación de ventas para registrar devoluciones, la separación de Nequi en la caja o el consentimiento para el tratamiento de datos— surgieron de las revisiones y del análisis de cumplimiento, y pudieron incorporarse sin rehacer lo construido gracias a la arquitectura por capas. Como aprendizaje, se evidenció que tareas aparentemente simples, como la edición de una venta ya registrada, exigen un análisis cuidadoso de sus efectos sobre el stock, la caja y la trazabilidad.

La estrategia de pruebas automatizadas, ejecutada en integración continua, fue una de las prácticas de mayor valor: las 160 pruebas actuaron como red de seguridad en cada cambio y como documentación ejecutable de las reglas del negocio. Las pruebas de rendimiento mostraron tiempos muy inferiores a los exigidos por los requisitos. Queda como observación que la validación en operación real con el personal de la tienda es la que finalmente permite medir la reducción de pérdidas y de tiempos operativos planteada en el anteproyecto; el sistema provee los indicadores necesarios para hacerlo de forma objetiva. [COMPLETAR: conclusión sobre los resultados de la encuesta y de la operación real].

7. Recomendaciones

Para la tienda MundoCoco se recomienda mantener la disciplina de registrar cada venta y cada gasto en el momento en que ocurre y cerrar la caja todos los días, pues la calidad de los reportes depende de ello. Se sugiere revisar semanalmente los indicadores de gestión —en especial la precisión del inventario y las cajas con diferencia— y realizar conteos físicos periódicos para registrar los ajustes correspondientes. También es importante verificar que la tarea programada de respaldo se ejecute en el servidor definitivo y guardar copias fuera del equipo.

Como mejoras futuras del sistema se proponen: la facturación electrónica ante la DIAN; un módulo de compras con proveedores y órdenes de compra a partir de la sugerencia de reorden; el control de fechas de vencimiento por lote para los productos perecederos; el registro de clientes y programas de fidelización; la lectura de códigos de barras en el punto de venta; y la activación de varias sucursales, para la cual el modelo de datos ya está preparado. Finalmente, se recomienda aprovechar la API de integración para conectar el sistema con el software contable del propietario.

Referencias

- Amaya Beltrán, J., y Preciado Guerrero, M. (2013). *Desarrollo de una solución web de optimización de procesos de inventario y cotización para la perfumería El Mundo de Tu Belleza* [Trabajo de grado, Universidad EAN].
<https://repository.universidadean.edu.co/entities/publication/7802bd36-575c-4f0f-82eb-56f6d2d1f0df>
- Ballou, R. H. (2004). *Logística: Administración de la cadena de suministro* (5.^a ed.). Pearson Educación.
- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., y Thomas, D. (2001). *Manifiesto por el desarrollo ágil de software*. <https://agilemanifesto.org/>
- Booch, G., Rumbaugh, J., y Jacobson, I. (1999). *The Unified Modeling Language user guide*. Addison-Wesley.
- Bowersox, D. J., Closs, D. J., y Cooper, M. B. (2007). *Supply chain logistics management* (2.^a ed.). McGraw-Hill.
- Chen, P. P.-S. (1976). The entity-relationship model: Toward a unified view of data. *ACM Transactions on Database Systems*, 1(1), 9–36. <https://doi.org/10.1145/320434.320440>
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387. <https://doi.org/10.1145/362384.362685>
- Congreso de Colombia. (1982, 28 de enero). *Ley 23 de 1982, sobre derechos de autor*.
<https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=3431>

Congreso de Colombia. (1999, 18 de agosto). *Ley 527 de 1999, por medio de la cual se define y reglamenta el acceso y uso de los mensajes de datos, del comercio electrónico y de las firmas digitales.*

<https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=4276>

Congreso de Colombia. (2000, 27 de julio). *Ley 603 de 2000, por la cual se modifica el artículo 47 de la Ley 222 de 1995.*

Congreso de Colombia. (2008, 31 de diciembre). *Ley Estatutaria 1266 de 2008, por la cual se dictan las disposiciones generales del hábeas data.*

Congreso de Colombia. (2012, 17 de octubre). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales.*

<https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981>

Filament. (s.f.). *Filament documentation*. Recuperado de <https://filamentphp.com/docs>

Fowler, M. (2004). *UML distilled: A brief guide to the standard object modeling language* (3.^a ed.). Addison-Wesley.

Gitman, L. J., y Zutter, C. J. (2012). *Principios de administración financiera* (12.^a ed.). Pearson Educación.

Hernández, L. F., Rodríguez, M. P., y Gómez, S. A. (2021). Diseño e implementación de un sistema de gestión de inventarios para la empresa Técnitaller S.A.S. *Revista Ingeniería y Desarrollo*. <https://www.redalyc.org/journal/816/81661270007/html/>

International Organization for Standardization. (2017). *ISO/IEC/IEEE 12207:2017. Systems and software engineering — Software life cycle processes*. ISO.

International Organization for Standardization. (2018). *ISO/IEC/IEEE 29148:2018. Systems and software engineering — Life cycle processes — Requirements engineering*. ISO.

- International Organization for Standardization. (2022). *ISO/IEC/IEEE 29119-1:2022. Software and systems engineering — Software testing — Part 1: General concepts*. ISO.
- International Organization for Standardization. (2023). *ISO/IEC 25010:2023. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. ISO.
- Kaner, C., Falk, J., y Nguyen, H. Q. (1999). *Testing computer software* (2.^a ed.). Wiley.
- Laravel. (s.f.). *Laravel documentation*. Recuperado de <https://laravel.com/docs>
- Laudon, K. C., y Laudon, J. P. (2020). *Sistemas de información gerencial* (16.^a ed.). Pearson.
- Myers, G. J., Sandler, C., y Badgett, T. (2011). *The art of software testing* (3.^a ed.). Wiley.
- Nielsen, J. (1994). *10 usability heuristics for user interface design*. Nielsen Norman Group.
<https://www.nngroup.com/articles/ten-usability-heuristics/>
- Oracle. (s.f.). *MySQL 8.0 reference manual*. Recuperado de
<https://dev.mysql.com/doc/refman/8.0/en/>
- Pest. (s.f.). *Pest documentation*. Recuperado de <https://pestphp.com/docs>
- Presidencia de la República de Colombia. (2013, 27 de junio). *Decreto 1377 de 2013, por el cual se reglamenta parcialmente la Ley 1581 de 2012*.
- Pressman, R. S., y Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9.^a ed.). McGraw-Hill.
- Schwaber, K., y Sutherland, J. (2020). *La guía de Scrum*. <https://scrumguides.org/>
- Sommerville, I. (2011). *Ingeniería de software* (9.^a ed.). Pearson Educación.
- Spatie. (s.f.). *Laravel-permission documentation*. Recuperado de <https://spatie.be/docs/laravel-permission>
- The PHP Group. (s.f.). *Manual de PHP*. Recuperado de <https://www.php.net/manual/es/>

- Universidad Católica de Colombia. (2017). *Desarrollo de sistema web para control de inventario en el área de soporte técnico de Arturo Calle S.A.S.* [Trabajo de grado].
<https://repository.ucatolica.edu.co/bitstream/10983/14503/1/DocumentoTrabajoDeGrado.pdf>
- Waters, D. (2003). *Inventory control and management* (2.^a ed.). Wiley.

Anexos

Anexo 1. Guía de preguntas para el levantamiento de información

Las siguientes preguntas orientaron las entrevistas con la administración y el personal de MundoCoco:

1. Describa de manera general cómo se realiza hoy el control del inventario, de las ventas y de la caja.
2. ¿Cuál es la principal dificultad que enfrentan con el proceso actual?
3. ¿Quién realiza el inventario y en qué momento de la jornada? ¿Cuánto tiempo toma?
4. ¿Con qué frecuencia se presentan diferencias en el cuadre de caja? ¿Cómo se resuelven?
5. ¿Qué formatos utilizan hoy (hojas, cuadernos, Excel)? ¿Pueden compartirlos?
6. ¿Cuántas personas usarán el sistema y qué funciones cumple cada una?
7. ¿Qué acciones debe poder realizar cada tipo de usuario? ¿Quién debe poder ver los reportes?
8. ¿Qué categorías y productos maneja la tienda? ¿Qué rango de precios tienen?
9. ¿Qué información es importante guardar de cada producto (código, precio de costo, stock mínimo, descripción)?
10. ¿Se puede vender un producto a un precio distinto al de lista? ¿En qué casos?
11. ¿Qué métodos de pago reciben (efectivo, Nequi, transferencias, tarjetas)?
12. ¿Cómo se registran hoy las devoluciones y los descuentos?
13. ¿Qué gastos se pagan con el dinero de la caja (materia prima, servicios, transporte)?
14. ¿Con qué base se abre la caja cada día y quién hace el cierre?
15. ¿Cómo se registran las compras a proveedores y las pérdidas por producto vencido o dañado?

16. ¿A partir de qué cantidad considera que un producto está por agotarse?
17. ¿Qué reportes le gustaría consultar (ventas por día, por producto, por categoría, productos más vendidos, inventario valorizado)?
18. ¿En qué formatos necesita descargar los reportes (PDF, Excel)?
19. ¿El sistema se usará desde un computador de la tienda, una tableta o un celular?
20. ¿Qué información del personal o de los clientes se almacenaría en el sistema?

Anexo 2. Encuesta de satisfacción del usuario final

Encuesta dirigida al personal de MundoCoco para evaluar la facilidad de uso y la utilidad del sistema al finalizar el periodo de pruebas en la tienda.

1. ¿Cuál es su rol dentro del sistema? (Administrador / Operario / Consultor)
2. ¿Qué tan fácil le resultó registrar una venta? (1 = Muy difícil, 5 = Muy fácil)
3. ¿El sistema descuenta correctamente el inventario al registrar las ventas? (Sí / No / A veces)
4. ¿Qué tan fácil le resultó abrir y cerrar la caja? (1 a 5)
5. ¿El cuadro de caja con el sistema es más rápido que el proceso anterior? (Sí / No / Igual)
6. ¿Le resultan útiles las alertas de productos con stock mínimo? (1 = Nada útiles, 5 = Muy útiles)
7. ¿La información de los reportes es clara y fácil de entender? (1 a 5)
8. ¿Ha tenido algún problema al registrar entradas o salidas de inventario? ¿Cuál?
9. ¿Los mensajes de error le permitieron entender y corregir el problema? (1 a 5)
10. ¿Cómo califica en general su experiencia con el sistema? (Muy mala / Mala / Regular / Buena / Muy buena)
11. ¿Qué funcionalidad le gustaría que se agregara o mejorara?

[COMPLETAR: adjuntar los resultados de la encuesta (gráficas o tabla de respuestas)]

Anexo 3. Carta de autorización de la empresa MundoCoco

Mediante carta fechada el 18 de noviembre de 2025, la administradora de MundoCoco, Jazmín Karina Meneses, autorizó formalmente a los estudiantes Cristian Camilo Díaz Ramírez y Carlos Daniel Camacho Valencia, del programa Tecnología en Sistemas de Información de la Institución Universitaria Antonio José Camacho, para desarrollar en la empresa el proyecto académico “Aplicativo de Gestión de Inventarios para Tienda MundoCoco”. En ella, la tienda se compromete a brindar la información necesaria, facilitar el levantamiento de requisitos y permitir las pruebas en su entorno real, y los estudiantes a utilizar los datos únicamente con fines académicos, garantizando su confidencialidad conforme a las normas de protección de datos personales.

CALI, 18 de noviembre de 2025

Señores:

Institución Universitaria Antonio José Camacho – UNIAJC

Programa de Tecnología en Sistemas de Información

Cali, Colombia.



Asunto: Autorización para desarrollo de proyecto académico

Yo, **Jazmín Karina Meneses**, identificado(a) con cédula de ciudadanía No. 31710982, en calidad de administradora(a) de la empresa **MundoCoco**, ubicada en la ciudad de Cali y dedicada a la venta de postres, helados y bebidas, me permito manifestar por medio de la presente que **autorizo formalmente** a los estudiantes:

- **Cristian Camilo Díaz Ramírez**, con cedula de ciudadanía No.1006100017
- **Carlos Daniel Camacho Valencia**, con cedula de ciudadanía No. 1110286798

del programa **Tecnología en Sistemas de Información**, para desarrollar en nuestra empresa el proyecto académico titulado:

“Aplicativo de Gestión de Inventarios para Tienda MundoCoco”.

Dicho proyecto busca analizar, diseñar y desarrollar una solución web que permita optimizar los procesos de control de inventarios, registro de ventas y cuadre de caja. MundoCoco reconoce la importancia de este tipo de iniciativas y se compromete a brindar la información necesaria, facilitar el levantamiento de requisitos y permitir las pruebas pertinentes en el entorno real de la tienda, siempre que no interfieran con la operación normal del negocio.

Así mismo, declaro que los estudiantes podrán utilizar los datos proporcionados únicamente con fines académicos, garantizando la confidencialidad de la información suministrada y el cumplimiento de las normas vigentes relacionadas con la protección de datos personales.

Agradezco a la institución por el acompañamiento académico brindado y expreso mi disposición para colaborar durante el desarrollo del proyecto.

Atentamente,

Jazmín Karina Meneses

Administradora – MundoCoco

C.C. 31710982

Celular: 315 589 3100

Anexo 4. Resumen del manual de usuario

El manual de usuario completo se entrega como documento independiente. Su contenido se resume en el recorrido diario recomendado:

Tabla 29. *Recorrido diario recomendado del manual de usuario*

Paso	Quién	Dónde
1. Abrir la caja con el dinero base	Administrador	Operación diaria → Cajas → Nueva caja
2. Registrar entradas de mercancía	Administrador	Catálogo → Movimientos → Nuevo movimiento
3. Registrar cada venta	Operario o Administrador	Operación diaria → Ventas → Nueva venta
4. Registrar los gastos del día	Administrador	Operación diaria → Gastos → Nuevo gasto
5. Revisar alertas e informes	Todos, según su rol	Escritorio e Informes → Reportes
6. Cerrar la caja contando el dinero	Administrador	Cajas → Cerrar caja

Nota. Tomado del manual de usuario del sistema (elaboración propia).

Regla principal: una fecha con la caja cerrada queda bloqueada; ya no se pueden registrar, editar ni anular ventas ni gastos de ese día.

Anexo 5. Matriz de trazabilidad entre requisitos y pruebas

Tabla 30. *Trazabilidad de requisitos, casos funcionales y pruebas automatizadas*

Requisito	Casos funcionales	Pruebas automatizadas
RF01	PF-05, PF-06, PF-07	ProductoTest, ProductoCodigoTest, CumplimientoAnteproyectoTest (borrado lógico)
RF02 / RF03	PF-08, PF-09	InventarioServiceTest, InventarioFlowTest, CumplimientoAnteproyectoTest (lotes)
RF04	PF-10 a PF-16	VentaServiceTest, VentasPanelTest, VentasFlowTest, VentaAlertaStockTest, CumplimientoAnteproyectoTest
RF05	PF-08	InventarioServiceTest, CumplimientoAnteproyectoTest (RF05)
RF06	—	CategoriaTest, SimulacionRealMundoCocoTest
RF07	PF-14	VentaAlertaStockTest, CumplimientoAnteproyectoTest (reorden)
RF08 / RF09	PF-21, PF-22	ReportesServiceTest, ReporteExportTest
RF10	PF-01 a PF-04	CumplimientoAnteproyectoTest (roles), AdminUserSeederTest
RF11	PF-17 a PF-20	CajaTest, CajaServiceTest, CajasFlowTest, CajaConGastosTest, GastoObserverTest
RF12	PF-23	TrazabilidadTest, ProductoObserverTest
RNF01 / RNF09	Pruebas de rendimiento	RendimientoTest, SimulacionRealMundoCocoTest
RNF04 / RNF05	PF-24	BackupDatabaseTest, VentasPanelTest (sin registros a medias)
RNF10 (Ley 1581)	PF-03	CumplimientoAnteproyectoTest (consentimiento)

Nota. Elaboración propia.